Data Science Capstone Project - Political Globalization Analysis

The Data Science Capstone Project involves analyzing the impact of various socio-economic indicators on the Political Globalization Index. To address this, we are using a dataset that includes a range of economic and demographic variables. The goal is to determine the relative importance of these indicators in influencing the Political Globalization Index and to gain insights into the complex relationship between political globalization and socio-economic factors.

Project Problem Statement: The project aims to understand the influence of various socio-economic indicators on the Political Globalization Index for different countries. The problem statement can be framed as follows:

"Given a dataset containing information on multiple socio-economic indicators such as GDP per capita, population size, unemployment rate, and others, the goal is to analyze and quantify the impact of these factors on the Political Globalization Index. By building a predictive model, we seek to determine the relative importance of these indicators in shaping political globalization and identify which socio-economic factors are the most influential in different countries."

Political Globalization Index Importance: The Political Globalization Index's significance lies in its reflection of how a country's internal political system interacts with socio-economic and demographic changes to influence its global political engagement. A nation's demographic shifts, such as population growth and age distribution, can impact political dynamics, with younger populations often seeking more participation in global affairs. Socio-economic factors like GDP per capita and political stability also shape a nation's stance on global diplomacy. Government policies, leadership changes, and citizen engagement further mold a country's international involvement. By examining these factors, we can better comprehend how a nation's internal political system reacts to socio-economic and demographic changes, ultimately affecting its Political Globalization Index and its role in the global political arena. This analysis is invaluable for policymakers and researchers aiming to understand and influence a nation's international relations.

To assess the importance of the Political Globalization Index:

1. We will investigate the relationship between political globalization and the selected socio-economic indicators.
2. Determine which indicators have the most significant impact on the Political Globalization Index.
3. Explore variations across countries and regions to identify patterns.
4. Develop a predictive model that can help forecast the Political Globalization Index based on the socio-economic indicators.
5. Understand the policy implications and potential drivers of political globalization in different countries.

By addressing this problem, we not only gain insights into the significance of the Political Globalization Index but also contribute to a better understanding of the complex dynamics between a nation's socio-economic situation and its political engagement on the global

Data Sources

https://datatopics.worldbank.org/world-development-indicators/ (https://datatopics.worldbank.org/world-development-indicators/)

- World Development Indicators (WDI) database from the World Bank. This database is a comprehensive source of data on global development and includes a wide range of indicators covering various aspects of economic, social, and environmental development.

https://www.imf.org/en/Home (https://www.imf.org/en/Home)

- The International Monetary Fund (IMF) is a major international financial institution that provides financial assistance, policy advice, and research on economic and financial issues to its member countries. The IMF's official website, located at https://www.imf.org/en/Home (https://www.imf.org/en/Home), serves as a comprehensive and authoritative source for various types of data, reports, research, and information related to the global economy.

https://www.theglobaleconomy.com/ (https://www.theglobaleconomy.com/)

- "The Global Economy" (https://www.theglobaleconomy.com/ (https://www.theglobaleconomy.com/)) is a comprehensive online platform that provides a wealth of economic data, statistics, analysis, and information related to the global economy and individual countries. This data source focuses on delivering economic indicators, forecasts, and insights for researchers, analysts, policymakers, and anyone interested in understanding the world's economic landscape.

In [3]:
```
# Importing Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

import plotly.express as px
import plotly.offline as py
from plotly.offline import download_plotlyjs, init_notebook_mode, iplo
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

In [4]:
```
import warnings
warnings.filterwarnings('ignore')
```

In [5]:
```
#pip install tabulate
```

In [6]:
```
# Load the pollution_dataset
df_EcoInd = pd.read_csv('C:\SPU\Team\Capstone\MacroEconomy_2000_2023_I
df_DemoInd = pd.read_csv('C:\SPU\Team\Capstone\Demographic_Health_Indi
df_DevInd = pd.read_csv('C:\SPU\Team\Capstone\Developement_Indicators.
```

Dataset Description:

df_EcoInd - Economic and Financial Indicators:

> - This dataset provides a wide range of economic and financial indicators, which can be used for economic analysis, policy evaluation, and comparisons between countries and regions. It is valuable for understanding the economic performance and financial stability of different countries over time.

The dataset you've provided contains information related to various economic and financial indicators for different countries over multiple years. Here's a brief description of the dataset columns:

1. 'Country': The name of the country.
2. 'Code': A unique code or identifier for each country.

3. 'ContinentCode': A code or identifier for the continent to which the country belongs.
4. 'Year': The year to which the data pertains.
5. 'GDP per capita current U.S. dollars': The Gross Domestic Product (GDP) per capita in current U.S. dollars, which represents the economic output per person.
6. 'GDP per capita Purchasing Power Parity': The GDP per capita adjusted for purchasing power parity, which accounts for differences in price levels between countries.
7. 'Capital investment as percent of GDP': The percentage of the GDP that is invested in capital goods, such as machinery, infrastructure, and technology.
8. 'Inflation: percent change in the Consumer Price Index': The rate of inflation, expressed as the percentage change in the Consumer Price Index, which measures the average price change of a basket of goods and services over time.
9. 'Unemployment rate': The percentage of the labor force that is unemployed and actively seeking employment.
10. 'Labor force participation rate': The percentage of the working-age population that is either employed or actively seeking employment.
11. 'Exports of goods and services as a percent of GDP': The total value of a country's exports of goods and services as a percentage of its GDP.
12. 'Imports of goods and services as a percent of GDP': The total value of a country's imports of goods and services as a percentage of its GDP.
13. 'Foreign Direct Investment percent of GDP': The percentage of a country's GDP that comes from foreign direct investment.
14. 'Current account balance as percent of GDP': The balance of a country's international trade in goods and services, expressed as a percentage of its GDP.
15. 'Trade balance as percent of GDP': The balance of a country's trade (exports minus imports) expressed as a percentage of its GDP.
16. 'Remittances as percent of GDP': The total value of remittances (money sent by individuals working abroad) as a percentage of a country's GDP.
17. 'Government spending as percent of GDP': The total government expenditure as a percentage of a country's GDP.
18. 'Fiscal balance percent of GDP': The balance between government revenues and government spending, expressed as a percentage of GDP.
19. 'Government debt as percent of GDP': The total government debt as a percentage of a country's GDP.
20. 'Agriculture value added billion USD': The value added by the agriculture sector in billion U.S. dollars.
21. 'Liquid liabilities percent of GDP': The percentage of a country's GDP represented by liquid liabilities, which typically include cash, short-term investments, and accounts payable.
22. 'Bank assets percent of GDP': The total assets held by banks as a percentage of a country's GDP.

df_DemoInd - Demographic and Healthcare Indicators:

- This dataset provides valuable information about the demographic profile and healthcare conditions of different countries. Researchers, policymakers, and analysts often use such data to study and compare the socio-economic development, health, and education-related

The dataset contains a wide range of demographic and healthcare-related indicators for various countries. Here's a description of the columns in the dataset:

1. 'Population size in millions': The total population of a country expressed in millions.
2. 'Percent urban population': The percentage of the country's population residing in urban areas as opposed to rural areas.
3. 'Population density people per square km': The number of people per square kilometer in the country, indicating population concentration.
4. 'Female population percent of total': The percentage of the population that is female, relative to the total population.
5. 'Rural population percent of total population': The percentage of the total population living in rural areas.
6. 'Population growth percent': The rate at which the population is growing, expressed as a percentage.
7. 'Health spending per capita': The amount of money spent on healthcare per person in the country.
8. 'Health spending as percent of GDP': The percentage of the country's GDP allocated for healthcare and medical expenses.
9. 'Life expectancy in years': The average number of years a person in the country can expect to live.
10. 'Death rate per 1000 people': The number of deaths per 1000 people in the country, often used as a measure of overall health and well-being.
11. 'Fertility rate births per woman': The average number of children a woman is expected to have during her lifetime.
12. 'Maternal mortality per 100,000 live births': The number of maternal deaths per 100,000 live births, indicating the risk of maternal mortality during childbirth.
13. 'Public spending on education percent of GDP': The percentage of a country's GDP allocated for public education.
14. 'Public spending on education percent of public spending': The percentage of a country's total public spending dedicated to education.
15. 'Percent of world population': The percentage of the world's total population represented by the country.

df_DevInd - Development and Global Indicators:

- These indices are valuable for understanding a country's position in the global context. They can be used to compare countries, assess development progress, and evaluate the impact of globalization on different aspects of a country's well-being. Researchers, policymakers, and international organizations often use these indices to make informed decisions and set development goals.

The dataset contains several indices that provide insights into various aspects of a country's development and global integration. Here's a description of the columns in the dataset:

1. 'Globalization Index': This index quantifies the overall degree of globalization for a country. It takes into account economic, political, and social globalization factors to provide a holistic view of a country's global integration.
2. 'Economic Globalization Index': This index specifically measures the economic aspects of globalization. It considers factors such as trade openness, foreign direct investment, and economic interaction with the rest of the world.
3. 'Political Globalization Index': This index assesses a country's political engagement with the international community. It may include factors like diplomatic relations, participation in international organizations, and political cooperation.
4. 'Social Globalization Index': The Social Globalization Index focuses on social and cultural aspects of globalization. It considers factors such as international travel, communication, and cultural exchange.
5. 'Human Development Index': The Human Development Index (HDI) is a composite index that assesses a country's overall human development. It combines indicators related to health (life expectancy), education (e.g., literacy and school enrollment), and standard of living (GDP per capita). HDI provides a broader picture of a country's development beyond just economic or globalization aspects.

Displaying the First 3 Rows of the each dataset

In [7]:
```
df_EcoInd.head(3)
```

Out[7]:

	Country	Code	ContinentCode	Year	GDP per capita current U.S. dollars	GDP per capita Purchasing Power Parity	Capital investment as percent of GDP	Inflation percen change ir the Consume Price Index
0	Afghanistan	AFG	Asia	2000	157.65	1271.63	11.5	10.4(
1	Afghanistan	AFG	Asia	2001	176.48	1276.50	9.8	8.67
2	Afghanistan	AFG	Asia	2002	183.53	1280.46	12.6	6.4(

3 rows × 22 columns

In [8]:
```
df_DemoInd.head(3)
```

Out[8]:

	Country	Code	ContinentCode	Year	Population size in millions	Percent urban population	Population density people per square km	Fem popula percen t
0	Afghanistan	AFG	Asia	2000	19.54	22.08	30.0	4
1	Afghanistan	AFG	Asia	2001	19.69	22.17	30.0	4
2	Afghanistan	AFG	Asia	2002	21.00	22.26	32.0	4

In [9]:
```
df_DevInd.head(3)
```

Out[9]:

	Country	Year	Globalization index (0-100)	Economic globalization index (0-100)	Political globalization index (0-100)	Social globalization index (0-100)	Happiness Index 0 (unhappy) - 10 (happy)
0	Afghanistan	2000	24.55	36.25	25.53	11.08	NaN
1	Afghanistan	2001	24.51	36.00	25.41	11.43	NaN
2	Afghanistan	2002	25.06	35.61	25.66	13.42	NaN

Statistical Analysis of the datsets

In [10]:
```
#Statistical Analysis
df_EcoInd.describe()
```

Out[10]:

	Year	GDP per capita current U.S. dollars	GDP per capita Purchasing Power Parity	Capital investment as percent of GDP	Inflation: percent change in the Consumer Price Index	Unemplo
count	4549.000000	4379.000000	4219.000000	3807.000000	3993.000000	4135.0
mean	2010.987909	14677.213224	19811.346900	23.986336	6.325212	8.0
std	6.627584	24017.313788	21636.191109	8.452747	18.794927	6.0
min	2000.000000	110.460000	628.690000	-3.950000	-16.900000	0.0
25%	2005.000000	1422.510000	4135.205000	18.950000	1.600000	3.7
50%	2011.000000	4722.040000	11856.050000	22.970000	3.500000	6.3
75%	2017.000000	17229.300000	29282.230000	27.695000	7.000000	10.6
max	2022.000000	234317.080000	157602.480000	79.400000	557.200000	37.3

In [11]:
```
#Statistical Analysis
df_DemoInd.describe()
```

Out[11]:

	Year	Population size in millions	Percent urban population	Population density people per square km	Female population percent of total	Rural population percent of total population
count	4508.000000	4508.000000	4508.000000	4116.000000	4508.000000	4508.000000
mean	2011.000000	35.777065	57.564616	416.651603	50.018578	42.435435
std	6.633985	135.624024	23.570314	1931.633594	2.951142	23.570318
min	2000.000000	0.010000	8.250000	2.000000	23.390000	0.000000
25%	2005.000000	1.530000	38.362500	31.000000	49.680000	23.325000
50%	2011.000000	7.040000	57.560000	79.000000	50.350000	42.440000
75%	2017.000000	24.600000	76.675000	190.000000	51.070000	61.637500
max	2022.000000	1417.170000	100.000000	21595.000000	55.040000	91.750000

In [12]:
```
#Statistical Analysis
df_DevInd.describe()
```

Out[12]:

	Year	Globalization index (0-100)	Economic globalization index (0-100)	Political globalization index (0-100)	Social globalization index (0-100)	Happines Index (unhappy) 10 (happy
count	4414.000000	4033.000000	3881.000000	4080.000000	4080.000000	1333.00000
mean	2010.821477	59.216844	57.628771	60.868787	59.889922	5.44603
std	6.537544	15.241773	15.812703	23.908979	19.607143	1.12090
min	2000.000000	24.500000	20.400000	2.280000	10.880000	1.86000
25%	2005.000000	47.660000	46.000000	43.620000	44.177500	4.58000
50%	2011.000000	57.150000	57.240000	64.000000	62.110000	5.43000
75%	2016.000000	70.200000	68.700000	79.900000	76.610000	6.27000
max	2022.000000	91.310000	95.290000	98.590000	92.270000	7.84000

Data Analysis

Understanding Missing values

In [13]:
```
def missing_data_info(df):
    missing_count = df.isna().sum() #Checking for null values
    missing_count = pd.DataFrame(missing_count, columns=['Count'])
    missing_count = missing_count.reset_index().rename(columns={'index
    missing_count['percent'] = (missing_count.Count/len(df))*100
    missing_count = missing_count.sort_values('Count', ascending=False
    return missing_count
```

In [14]:
```
mdi_eco = missing_data_info(df_EcoInd)
mdi_eco.style.background_gradient(cmap='Blues')
```

Out[14]:

	Column	Count	percent
0	Liquid liabilities percent of GDP	827	18.179820
1	Fiscal balance percent of GDP	810	17.806111
2	Bank assets percent of GDP	772	16.970763
3	Government spending as percent of GDP	764	16.794900
4	Capital investment as percent of GDP	742	16.311277
5	Government debt as percent of GDP	683	15.014289
6	Exports of goods and services as percent of GDP	656	14.420752
7	Trade balance as percent of GDP	656	14.420752
8	Imports of goods and services as percent of GDP	656	14.420752
9	Inflation: percent change in the Consumer Price Index	556	12.222466
10	Remittances as percent of GDP	545	11.980655
11	Current account balance as percent of GDP	492	10.815564
12	Labor force participation rate	415	9.122884
13	Unemployment rate	414	9.100901
14	Foreign Direct Investment percent of GDP	376	8.265553
15	Agriculture value added billion USD	370	8.133656
16	GDP per capita Purchasing Power Parity	330	7.254342
17	GDP per capita current U.S. dollars	170	3.737085
18	Code	0	0.000000
19	Year	0	0.000000
20	ContinentCode	0	0.000000
21	Country	0	0.000000

In [15]:
```
mdi_demo = missing_data_info(df_DemoInd)
mdi_demo.style.background_gradient(cmap='Blues')
```

Out[15]:

	Column	Count	percent
0	Public spending on education percent of GDP	1525	33.828749
1	Public spending on education percent of public spending	1444	32.031943
2	Maternal mortality per 100000 live births	686	15.217391
3	Health spending as percent of GDP	675	14.973381
4	Health spending per capita	674	14.951198
5	Percent of world population	392	8.695652
6	Population density people per square km	392	8.695652
7	Life expectancy in years	282	6.255546
8	Fertility rate births per woman	273	6.055901
9	Death rate per 1000 people	229	5.079858
10	Population growth percent	196	4.347826
11	Code	0	0.000000
12	Rural population percent of total population	0	0.000000
13	Female population percent of total	0	0.000000
14	Percent urban population	0	0.000000
15	Population size in millions	0	0.000000
16	Year	0	0.000000
17	ContinentCode	0	0.000000
18	Country	0	0.000000

In [16]:
```
mdi_dev = missing_data_info(df_DevInd)
mdi_dev.style.background_gradient(cmap='Blues')
```

Out[16]:

	Column	Count	percent
0	Happiness Index 0 (unhappy) - 10 (happy)	3081	69.800634
1	Economic globalization index (0-100)	533	12.075215
2	Human Development Index (0 - 1)	414	9.379248
3	Globalization index (0-100)	381	8.631627
4	Political globalization index (0-100)	334	7.566833
5	Social globalization index (0-100)	334	7.566833
6	Country	0	0.000000
7	Year	0	0.000000

Renamig the columns

In [17]:
```
# Renamig the columns
df_DevInd.rename(columns={
    'Happiness Index 0 (unhappy) - 10 (happy)' : 'Happiness Index',
    'Globalization index (0-100)': 'Globalization Index',
    'Economic globalization index (0-100)': 'Economic Globalization In
    'Political globalization index (0-100)': 'Political Globalization
    'Social globalization index (0-100)': 'Social Globalization Index'
    'Human Development Index (0 - 1)': 'Human Development Index'
}, inplace=True)
```

Filling the missing values with the mean values of the respective country of the feature

In [18]:
```
# Group the DataFrame by 'Country', 'Code', and 'ContinentCode'
grouped = df_EcoInd.groupby(['Country', 'Code', 'ContinentCode'])

# Iterate through each group and fill missing values with the mean
for column in df_EcoInd.columns:
    if column != 'Country' and column != 'Year' and column != 'Code' a
        df_EcoInd[column] = grouped[column].transform(lambda x: x.fill

# Replace any remaining NaN values with 0
df_EcoInd.fillna(0, inplace=True)
```

In [19]:
```
# Group the DataFrame by 'Country', 'Code', and 'ContinentCode'
grouped = df_DemoInd.groupby(['Country', 'Code', 'ContinentCode'])

# Iterate through each group and fill missing values with the mean
for column in df_DemoInd.columns:
    if column != 'Country' and column != 'Year' and column != 'Code' a
        df_DemoInd[column] = grouped[column].transform(lambda x: x.fil

# Replace any remaining NaN values with 0
df_DemoInd.fillna(0, inplace=True)
```

In [20]:
```
# Group the DataFrame by 'Country', 'Code', and 'ContinentCode'
grouped = df_DevInd.groupby(['Country'])

# Iterate through each group and fill missing values with the mean
for column in df_DevInd.columns:
    if column != 'Country' and column != 'Year':  # Skip the 'Country'
        df_DevInd[column] = grouped[column].transform(lambda x: x.fill

# Replace any remaining NaN values with 0
df_DevInd.fillna(0, inplace=True)
```

This code performs group-wise imputation of missing values in the DataFrame. It calculates the mean for each column within each group and fills missing values with this mean.

1. **Grouping the DataFrame**:
   - The code groups the DataFrame by three columns: 'Country', 'Code', and 'ContinentCode'. This groups the data so that each group corresponds to a unique combination of these three columns.
2. **Iterating through each group and filling missing values with the mean**:
   - The code then iterates through each group.
   - For each column in the DataFrame (excluding 'Country', 'Year', 'Code', and 'ContinentCode'), it fills in missing (NaN) values with the mean of that column within the group. This means that within each group, missing values are filled with the mean of the available values for that column within that group.
3. **Replacing any remaining NaN values with 0**:
   - After filling missing values within groups, the code replaces any remaining NaN values with 0. This step is executed on the entire DataFrame, not just within groups.

Merging the datasets

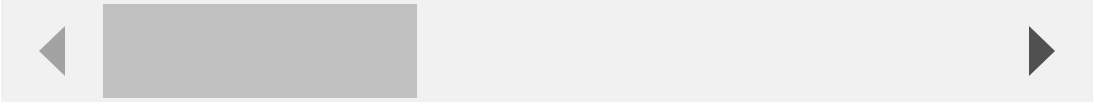
In [21]:
```
# Merge the datasets on common columns "Country", "Code", "ContinentCo
merged_df = df_EcoInd.merge(df_DemoInd, on=['Country', 'Code', 'Contin

merged_df = merged_df.merge(df_DevInd, on=['Country', 'Year'])
```

In [22]:
```
merged_df.head(3)
```

Out[22]:

	Country	Code	ContinentCode	Year	GDP per capita current U.S. dollars	GDP per capita Purchasing Power Parity	Capital investment as percent of GDP	Inflation percen change ir the Consume Price Index
0	Afghanistan	AFG	Asia	2000	157.65	1271.63	11.5	10.4(
1	Afghanistan	AFG	Asia	2001	176.48	1276.50	9.8	8.67
2	Afghanistan	AFG	Asia	2002	183.53	1280.46	12.6	6.4(

3 rows × 43 columns



In [23]:
```
merged_df.info
```

Out[23]:
```
<bound method DataFrame.info of          Country Code ContinentCode  Year  \
0     Afghanistan  AFG          Asia  2000
1     Afghanistan  AFG          Asia  2001
2     Afghanistan  AFG          Asia  2002
3     Afghanistan  AFG          Asia  2003
4     Afghanistan  AFG          Asia  2004
...           ...  ...           ...   ...
4356     Zimbabwe  ZWE        Africa  2018
4357     Zimbabwe  ZWE        Africa  2019
4358     Zimbabwe  ZWE        Africa  2020
4359     Zimbabwe  ZWE        Africa  2021
4360     Zimbabwe  ZWE        Africa  2022

      GDP per capita current U.S. dollars  \
0                                  157.65
1                                  176.48
2                                  183.53
3                                  200.46
4                                  221.66
...                                   ...
4356                              2269.18
4357                              1421.87
4358                              1372.70
4359                              1773.92
4360                              1267.00

      GDP per capita Purchasing Power Parity  \
0                                    1271.63
1                                    1276.50
2                                    1280.46
3                                    1292.33
4                                    1260.06
...                                      ...
4356                                 2399.62
4357                                 2203.40
4358                                 1990.32
4359                                 2115.14
4360                                 2143.24

      Capital investment as percent of GDP  \
0                                11.500000
1                                 9.800000
2                                12.600000
3                                11.800000
4                                13.500000
...                                    ...
4356                             14.150000
4357                             13.800000
4358                             13.150000
4359                             15.500000
4360                             10.023182

      Inflation: percent change in the Consumer Price Index  \
0                                                     10.40
1                                                      8.67
2                                                      6.40
```

```
3                                                  7.80
4                                                  8.30
...                                                 ...
4356                                              10.60
4357                                             255.30
4358                                             557.20
4359                                              98.50
4360                                             104.70

      Unemployment rate  Labor force participation rate  ...  \
0                  8.05                           46.76  ...
1                  8.04                           46.79  ...
2                  8.19                           46.84  ...
3                  8.12                           46.91  ...
4                  8.05                           46.98  ...
...                 ...                             ...  ...
4356               6.78                           65.87  ...
4357               7.37                           65.79  ...
4358               7.90                           65.17  ...
4359               8.07                           65.59  ...
4360               7.95                           66.06  ...

      Maternal mortality per 100000 live births  \
0                                   1346.000000
1                                   1273.000000
2                                   1277.000000
3                                   1196.000000
4                                   1115.000000
...                                         ...
4356                                 359.000000
4357                                 393.000000
4358                                 357.000000
4359                                 498.619048
4360                                 498.619048

      Public spending on education percent of GDP  \
0                                         3.27375
1                                         3.27375
2                                         3.27375
3                                         3.27375
4                                         3.27375
...                                           ...
4356                                      3.87000
4357                                      5.09000
4358                                      5.09000
4359                                      5.09000
4360                                      5.09000

      Public spending on education percent of public spending  \
0                                                    13.670000
1                                                    13.670000
2                                                    13.670000
3                                                    13.670000
4                                                    13.670000
...                                                        ...
4356                                                 19.040000
4357                                                 22.838889
```

```
4358                                          15.670000
4359                                          22.838889
4360                                          22.838889

      Percent of world population  Globalization Index  \
0                        0.320000            24.550000
1                        0.320000            24.510000
2                        0.340000            25.060000
3                        0.360000            26.350000
4                        0.370000            27.030000
...                           ...                  ...
4356                     0.200000            51.780000
4357                     0.200000            53.020000
4358                     0.200000            53.000000
4359                     0.191905            49.309524
4360                     0.191905            49.309524

      Economic Globalization Index  Political Globalization Index  \
0                        36.250000                          25.53
1                        36.000000                          25.41
2                        35.610000                          25.66
3                        35.010000                          27.04
4                        36.180000                          27.17
...                            ...                            ...
4356                     38.100000                          69.16
4357                     41.220000                          68.98
4358                     40.000000                          69.00
4359                     36.664762                          67.47
4360                     36.664762                          67.47

      Social Globalization Index  Happiness Index  Human Development
Index
0                      11.080000         3.153333                 0.3
45000
1                      11.430000         3.153333                 0.3
47000
2                      13.420000         3.153333                 0.3
78000
3                      16.550000         3.153333                 0.3
87000
4                      17.470000         3.153333                 0.4
00000
...                          ...              ...
...
4356                   48.090000         3.690000                 0.5
63000
4357                   48.860000         3.660000                 0.5
63000
4358                   50.000000         3.150000                 0.5
71000
4359                   43.792381         3.000000                 0.5
93000
4360                   43.792381         3.200000                 0.4
93273

[4361 rows x 43 columns]>
```

In [24]:
```
merged_df.shape
```

Out[24]:
```
(4361, 43)
```

In [25]:
```
merged_df.columns
```

Out[25]:
```
Index(['Country', 'Code', 'ContinentCode', 'Year',
       'GDP per capita current U.S. dollars',
       'GDP per capita Purchasing Power Parity',
       'Capital investment as percent of GDP',
       'Inflation: percent change in the Consumer Price Index',
       'Unemployment rate', 'Labor force participation rate',
       'Exports of goods and services as percent of GDP',
       'Imports of goods and services as percent of GDP',
       'Foreign Direct Investment percent of GDP',
       'Current account balance as percent of GDP',
       'Trade balance as percent of GDP', 'Remittances as percent of
GDP',
       'Government spending as percent of GDP',
       'Fiscal balance percent of GDP', 'Government debt as percent o
f GDP',
       'Agriculture value added billion USD',
       'Liquid liabilities percent of GDP', 'Bank assets percent of G
DP',
       'Population size in millions', 'Percent urban population',
       'Population density people per square km',
       'Female population percent of total',
       'Rural population percent of total population',
       'Population growth percent', 'Health spending per capita',
       'Health spending as percent of GDP', 'Life expectancy in year
s',
       'Death rate per 1000 people', 'Fertility rate births per woma
n',
       'Maternal mortality per 100000 live births',
       'Public spending on education percent of GDP',
       'Public spending on education percent of public spending',
       'Percent of world population', 'Globalization Index',
       'Economic Globalization Index', 'Political Globalization Inde
x',
       'Social Globalization Index', 'Happiness Index',
       'Human Development Index'],
      dtype='object')
```

In [26]:
```
merged_df.describe()
```

Out[26]:

	Year	GDP per capita current U.S. dollars	GDP per capita Purchasing Power Parity	Capital investment as percent of GDP	Inflation: percent change in the Consumer Price Index	Unemplo
count	4361.000000	4361.000000	4361.000000	4361.000000	4361.000000	4361.0
mean	2010.810135	14429.589131	18908.947268	22.386602	6.468228	7.5
std	6.537029	23692.305623	21532.569061	10.358118	18.797054	6.1
min	2000.000000	0.000000	0.000000	-3.950000	-16.900000	0.0
25%	2005.000000	1345.480000	3447.530000	17.780000	1.400000	3.3
50%	2011.000000	4606.890000	11029.140000	22.450000	3.300000	5.8
75%	2016.000000	16744.580000	27523.680000	27.520000	7.000000	10.2
max	2022.000000	204097.200000	157602.480000	79.400000	557.200000	37.3

8 rows × 40 columns

In [27]:
```
mdi_dev = missing_data_info(merged_df)
mdi_dev.style.background_gradient(cmap='Reds')
```

Out[27]:

	Column	Count	percent
0	Country	0	0.000000
1	Fertility rate births per woman	0	0.000000
2	Population density people per square km	0	0.000000
3	Female population percent of total	0	0.000000
4	Rural population percent of total population	0	0.000000
5	Population growth percent	0	0.000000
6	Health spending per capita	0	0.000000
7	Health spending as percent of GDP	0	0.000000
8	Life expectancy in years	0	0.000000
9	Death rate per 1000 people	0	0.000000
10	Maternal mortality per 100000 live births	0	0.000000
11	Population size in millions	0	0.000000
12	Public spending on education percent of GDP	0	0.000000
13	Public spending on education percent of public spending	0	0.000000
14	Percent of world population	0	0.000000
15	Globalization Index	0	0.000000
16	Economic Globalization Index	0	0.000000
17	Political Globalization Index	0	0.000000
18	Social Globalization Index	0	0.000000
19	Happiness Index	0	0.000000
20	Percent urban population	0	0.000000
21	Bank assets percent of GDP	0	0.000000
22	Code	0	0.000000
23	Exports of goods and services as percent of GDP	0	0.000000
24	ContinentCode	0	0.000000
25	Year	0	0.000000
26	GDP per capita current U.S. dollars	0	0.000000
27	GDP per capita Purchasing Power Parity	0	0.000000
28	Capital investment as percent of GDP	0	0.000000
29	Inflation: percent change in the Consumer Price Index	0	0.000000
30	Unemployment rate	0	0.000000
31	Labor force participation rate	0	0.000000
32	Imports of goods and services as percent of GDP	0	0.000000
33	Liquid liabilities percent of GDP	0	0.000000
34	Foreign Direct Investment percent of GDP	0	0.000000
35	Current account balance as percent of GDP	0	0.000000

	Column	Count	percent
36	Trade balance as percent of GDP	0	0.000000
37	Remittances as percent of GDP	0	0.000000
38	Government spending as percent of GDP	0	0.000000
39	Fiscal balance percent of GDP	0	0.000000
40	Government debt as percent of GDP	0	0.000000
41	Agriculture value added billion USD	0	0.000000
42	Human Development Index	0	0.000000

Observations

- Since we have already filled the null values in the individual datasets, we can observe no null values**

Checking for Duplicate values

In [28]:
```
# Specify the subset of columns for duplicate checking
subset_columns = ['Country', 'Code', 'ContinentCode', 'Year']

# Check for duplicate rows based on the specified subset
duplicate_rows = merged_df.duplicated(subset=subset_columns)

# Count the number of duplicate rows
num_duplicate_rows = duplicate_rows.sum()

# Display the duplicate rows or their count
if num_duplicate_rows > 0:
    duplicate_data = merged_df[duplicate_rows]
    print(f"Number of duplicate rows: {num_duplicate_rows}")
    print("Duplicate Rows:")
    print(duplicate_data)
else:
    print("No duplicate rows found.")
```

```
No duplicate rows found.
```

Observations

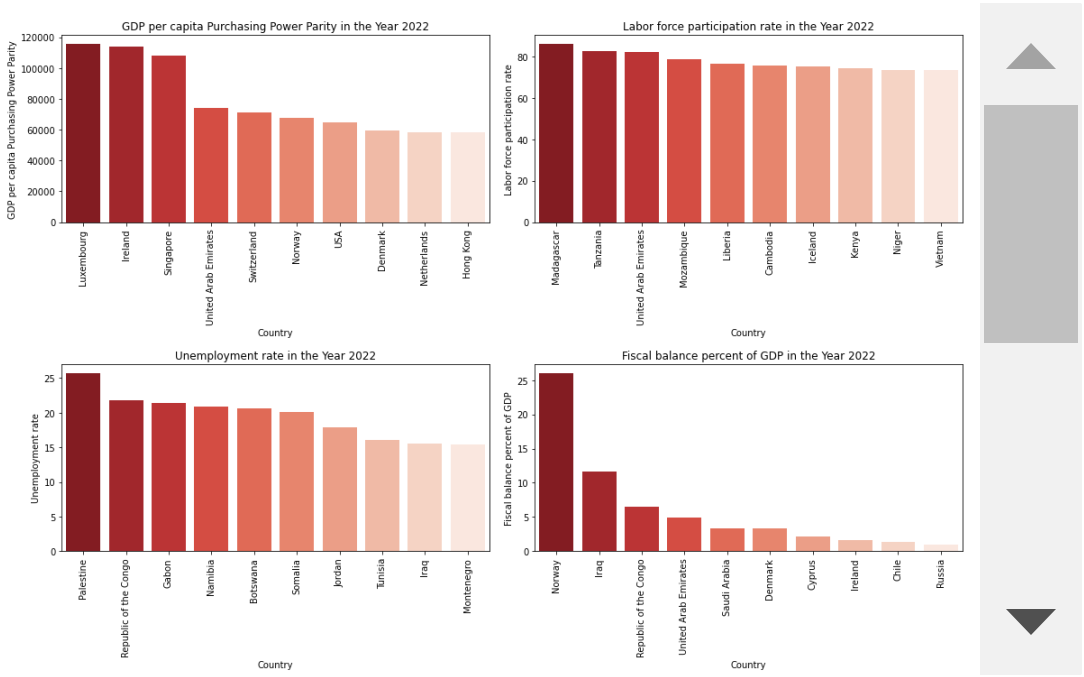
- No duplicate rows are found

EDA and Visualizations

2022 Socio Economic and Political Performance: Top 10 Rated Countries

In [29]:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Create a function for the plot
def economic_plot_2022(indicator, row, col):
    # Filter the DataFrame for the year 2022
    df_2022 = merged_df[merged_df['Year'] == 2022]

    # Sort the DataFrame by the specified economic indicator column an
    top_countries = df_2022[['Country', indicator]].sort_values(indica

    # Create a bar plot for the selected countries
    sns.barplot(data=top_countries, x='Country', y=indicator, palette=
    axes[row, col].set_xticklabels(top_countries['Country'], rotation=
    axes[row, col].set_title(f'{indicator} in the Year 2022')

# Create subplots in a 2x2 grid
fig, axes = plt.subplots(4, 2, figsize=(15, 20))

# Call the function for different economic indicators
economic_plot_2022('GDP per capita Purchasing Power Parity', 0, 0)
economic_plot_2022('Labor force participation rate', 0, 1)
economic_plot_2022('Unemployment rate', 1, 0)
economic_plot_2022('Fiscal balance percent of GDP', 1, 1)
economic_plot_2022('Health spending per capita', 2, 0)
economic_plot_2022('Foreign Direct Investment percent of GDP', 2, 1)
economic_plot_2022('Life expectancy in years', 3, 0)
economic_plot_2022('Political Globalization Index', 3, 1)


# Adjust layout and show the plots
plt.tight_layout()
plt.show()
```



Observations

GDP per capita Purchasing Power Parity

- Luxembourg has the highest GDP per capita Purchasing Power Parity at 115,541.77, followed by Ireland and Singapore.
- The top 10 countries in this category all have high GDP per capita values.
- Most of the top-performing nations, such as Luxembourg, Ireland, Switzerland, Norway, USA, Denmark, and the Netherlands, have showcased Political Globalization Index scores surpassing the average value of 71.69. This observation underscores that these countries, known for their economic prowess, also demonstrate a heightened level of commitment and participation in global political affairs, exceeding the typical engagement observed across all nations.

Labor force participation

- Madagascar has the highest labor force participation rate at 85.90%, followed by Tanzania and the United Arab Emirates.
- Labor force participation rates vary widely among these countries.
- This suggests that there is a diverse spectrum of political engagement among countries with high labor force participation rates, indicating that political globalization does not strictly correlate with this specific socio-economic indicator.

Unemployment rate

- Palestine has the highest unemployment rate at 25.72%, followed by the Republic of the Congo and Gabon.
- A higher Unemployment rate might be associated with reduced political engagement and connectivity at the global level due to the need for focused attention on domestic economic issues.

Fiscal balance percent of GDP

- Norway has the highest fiscal balance as a percentage of GDP at 26.04%, followed by Iraq and the Republic of the Congo.
- These countries maintain strong fiscal balances.
- The trend has multifaceted connection between fiscal balance and global political engagement.

Health spending per capita

- Switzerland has the highest health spending per capita at 7,477.40, followed by Norway and Luxembourg.
- These countries allocate significant resources to healthcare per capita.
- This suggests a general trend where countries with higher health spending also tend to have higher levels of political globalization.

Foreign Direct Investment percent of GDP

- Cyprus leads in foreign direct investment as a percentage of GDP with 39.14%, followed by Hong Kong and Singapore.
- These countries attract substantial foreign investments relative to their GDP.
- These top performers demonstrate a diverse range of political globalization.

Life expectancy in years

- Hong Kong has the highest life expectancy at 83.20 years, followed by Japan and Switzerland.
- These countries offer longer life expectancies to their populations.

Political Globalization Index

- France has the highest Political Globalization Index at 97.82, followed by Italy and the United Kingdom.
- These countries exhibit higher political globalization based on the index.

Average Political Globalization Index for the Year 2022

In [30]:
```
df_2022 = merged_df[merged_df['Year'] == 2022]

# Calculate the average Political Globalization Index for the year 202
average_pgi_2022 = df_2022['Political Globalization Index'].mean()
average_pgi_2022 ="{:.2f}".format(average_pgi_2022)
print(f"The average Political Globalization Index for the year 2022 is
```

```
The average Political Globalization Index for the year 2022 is: 71.69
```

Continent wise Mean GDP per capita PPP

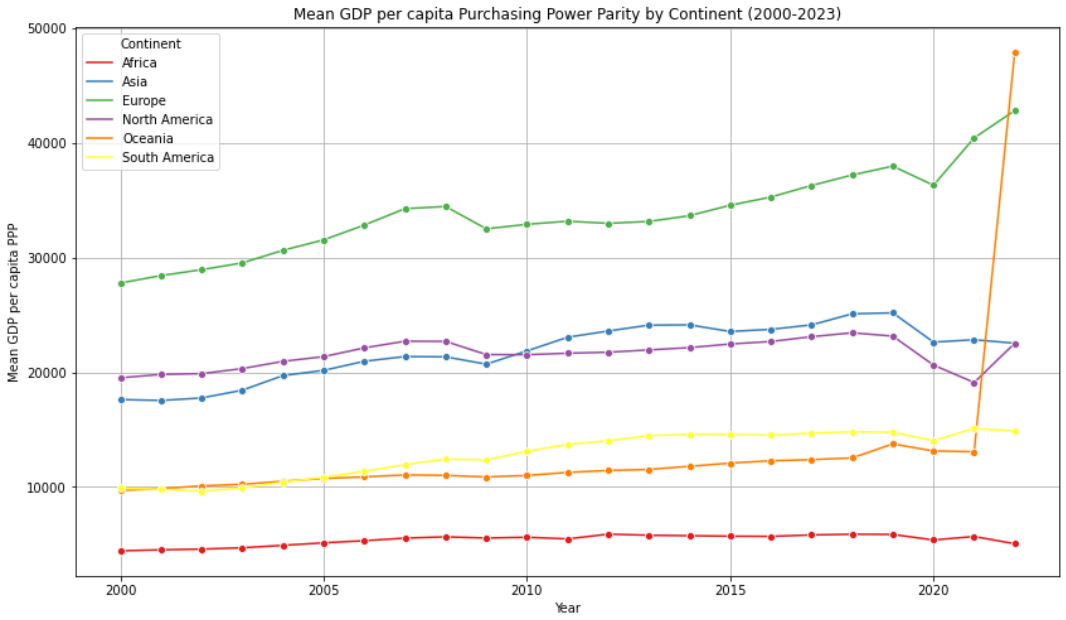
Importance of analyzing GDP per capita Purchasing Power Parity (PPP):

Analyzing GDP per capita PPP is a critical aspect of understanding a country's economic well-being, living standards, and development, with implications for various sectors, from policymaking to investment decisions.

- **Standard of Living:** GDP per capita PPP reflects the average income and the relative cost of living, providing insights into the standard of living within a country.
- **Economic Development:** It serves as an indicator of a country's level of economic development, with higher values indicating a more developed economy.
- **Quality of Life:** Higher GDP per capita PPP is associated with a better quality of life, including improved access to education, healthcare, and a higher level of consumption.
- **Economic Disparities:** It highlights income inequalities and disparities, which can have social and political implications.
- **Policy Decisions:** Governments use it to guide economic and social policy decisions, including taxation, social services, and infrastructure development.
- **Global Comparisons:** It allows for meaningful cross-country comparisons, adjusting for differences in price levels.
- **Economic Growth:** Changes in GDP per capita PPP over time reflect economic growth or decline, helping assess the effectiveness of economic policies.
- **Poverty Alleviation:** Lower GDP per capita PPP can signal higher poverty rates, which informs the design of poverty alleviation programs.
- **Economic Forecasting:** Economists and analysts use it to predict future economic trends, such as consumer spending and investment.

In [31]:

```
# Group the data by continent and year, and calculate the mean GDP per
mean_gdp_by_continent = merged_df.groupby(['ContinentCode', 'Year'])['

# Create a comparison graph for mean GDP per capita PPP by continent
plt.figure(figsize=(14, 8))
sns.lineplot(data=mean_gdp_by_continent, x='Year', y='GDP per capita P
plt.title('Mean GDP per capita Purchasing Power Parity by Continent (2
plt.xlabel('Year')
plt.ylabel('Mean GDP per capita PPP')
plt.legend(title='Continent')
plt.grid(True)

# Show the graph
plt.show()
```



observations - Mean GDP per capita Purchasing Power Parity by continent:

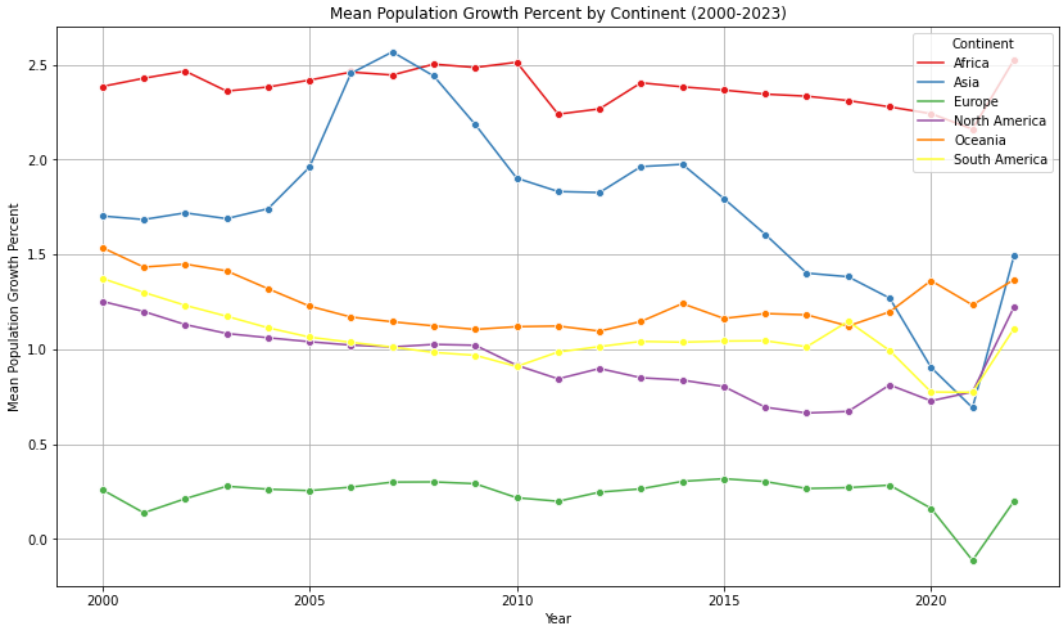
- **Europe**:
  - Europe had the highest mean GDP per capita PPP among all continents throughout this period.
  - It experienced steady growth, starting at approximately USD 27,786 in 2000 and reaching around USD 42,828 in 2022.
  - Europe consistently maintained a significant lead in GDP per capita PPP.
- **North America**:
  - North America started at around USD 19,523 in 2000, showing fluctuations in the following years.
  - In 2022, it experienced substantial growth, reaching a mean GDP per capita PPP of around USD 22,536.

    - North America's GDP per capita PPP is higher than most continents but lower than Europe.
- **Oceania**:
    - Oceania had a relatively stable GDP per capita PPP, starting at approximately USD 9,694 in 2000.
    - It exhibited a remarkable increase in 2022, reaching about USD 47,939.
    - While Oceania had a lower starting point, it experienced significant growth and had the highest increase from 2000 to 2022.
- **Asia**:
    - Asia started at around USD 17,626 in 2000 and showed consistent growth over the years.
    - It reached a peak of approximately USD 25,183 in 2019 and slightly decreased to about USD 22,536 in 2022.
    - Asia's GDP per capita PPP is substantial, making it one of the leading continents.
- **South America**:
    - South America experienced fluctuations in GDP per capita PPP, starting at around USD 9,885 in 2000.
    - It ended at approximately USD 14,884 in 2022, showing changes in economic stability.
    - South America's GDP per capita PPP remained lower than Europe, North America, and Asia.
- **Africa**:
    - Africa had the lowest mean GDP per capita PPP throughout this period.
    - It started at approximately USD 4,433 in 2000 and reached around USD 5,050 in 2022.
    - Africa's GDP per capita PPP is significantly lower compared to the other continents.

Europe consistently maintained the highest mean GDP per capita PPP, while Oceania exhibited the most significant growth over the years. North America and Asia also had substantial GDP per capita PPP, with North America experiencing notable growth in 2022. South America showed fluctuations, and Africa had the lowest mean GDP per capita PPP among the continents. These trends reflect the economic disparities and growth patterns in different regions of the world.

Continent wise Mean Population Growth Percent

In [32]:
```
# Group the data by continent and year, and calculate the mean Populat
mean_population_growth_by_continent = merged_df.groupby(['ContinentCod

# Create a comparison graph for mean Population Growth Percent by cont
plt.figure(figsize=(14, 8))
sns.lineplot(data=mean_population_growth_by_continent, x='Year', y='Po
plt.title('Mean Population Growth Percent by Continent (2000-2023)')
plt.xlabel('Year')
plt.ylabel('Mean Population Growth Percent')
plt.legend(title='Continent')
plt.grid(True)

# Show the graph
plt.show()
```



Observations:

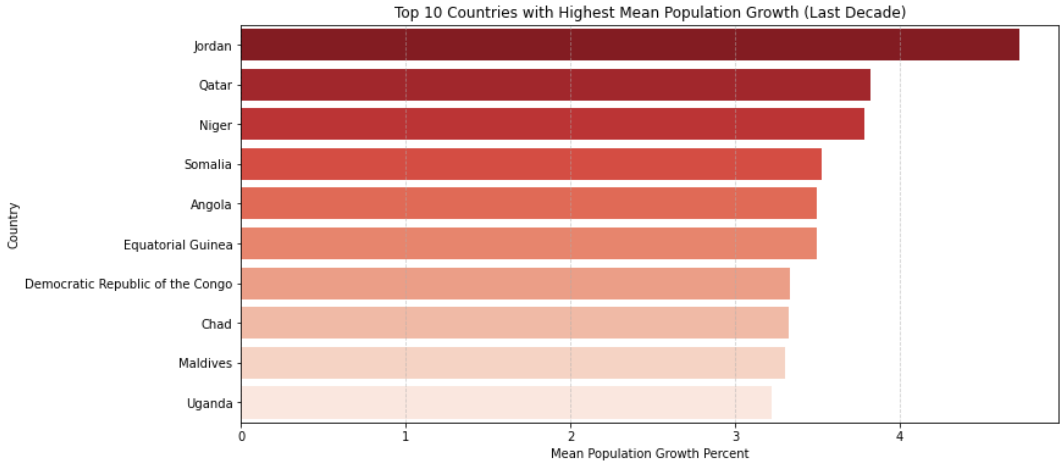
- **Africa**: Africa consistently maintains the highest population growth rate among all continents, with growth rates generally exceeding 2% throughout the observed period. This indicates a rapidly expanding population in the region.
- **Asia**: Asia follows as the second most populous continent, with a relatively high and consistent population growth rate typically ranging from 1.4% to 2%. This suggests continued population growth, albeit slightly slower than Africa.
- **Europe**: Europe experiences the lowest population growth rate among all continents, often hovering around or falling below 0.3%. This indicates that Europe has the slowest population growth compared to other continents.

- **North America**: North America, including the United States and Canada, maintains a moderate population growth rate, fluctuating around 1% during the observed period. It demonstrates a relatively steady and consistent growth pattern.
- **Oceania**: Oceania, encompassing Australia and nearby islands, follows a population growth rate similar to North America, hovering around 1% with some fluctuations. This indicates a relatively moderate population growth in this region.
- **South America**: South America exhibits varying population growth rates, typically ranging between 0.7% and 1.4%. While it experiences some fluctuations, it generally maintains a moderate population growth compared to other continents.

This provides a clearer picture of the population growth dynamics across continents, emphasizing the substantial variation between high-growth continents like Africa and Asia and low-growth regions such as Europe.

Top 10 Countries with Highest Mean Population Growth (Last Decade)

In [33]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate the start and end years for the last decade
start_year = 2013  # Current year minus 10
end_year = 2022  # Current year

# Filter the data for the last decade
last_decade_data = merged_df[(merged_df['Year'] >= start_year) & (merg

# Calculate the mean population growth by country for the last decade
mean_population_growth_last_decade = last_decade_data.groupby('Country

# Get the top 10 countries with the highest mean population growth
top_10_population_growth_last_decade = mean_population_growth_last_dec

# Create a vertical bar graph for the top 10 countries
plt.figure(figsize=(12, 6))
sns.barplot(data=top_10_population_growth_last_decade, x='Population g
plt.xlabel('Mean Population Growth Percent')
plt.ylabel('Country')
plt.title('Top 10 Countries with Highest Mean Population Growth (Last
plt.grid(axis='x', linestyle='--', alpha=0.6)

# Show the graph
plt.show()
```



Observations

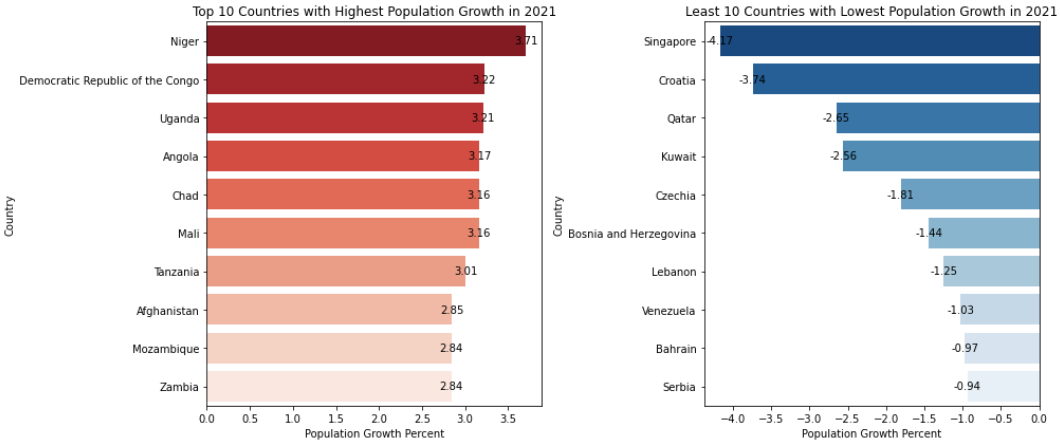
1. **Jordan (4.726045) and Qatar (3.822222):** Jordan and Qatar stand out as countries with the highest mean population growth rates, indicating significant demographic changes and substantial population increases over the last decade.

2. **Sub-Saharan African Nations:** Several countries from sub-Saharan Africa, including Niger, Somalia, Angola, and Equatorial Guinea, feature in the top 10, showcasing the region's notable population growth during this period.
3. **Diverse Influences:** These high population growth rates could be influenced by various factors, including birth rates, migration patterns, and economic opportunities, as well as the social and political stability of these regions.

Population Growth Percent 2021 - Top 10 and Least 10

In [34]:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Filter the data for the year 2021
population_growth_2021 = merged_df[merged_df['Year'] == 2021]

# Sort the data to get the top 10 countries with the highest populatio
top_10_population_growth_2021 = population_growth_2021[['Country', 'Po

# Sort the data to get the least 10 countries with the lowest populati
least_10_population_growth_2021 = population_growth_2021[['Country', '

# Create two subplots for the top 10 and least 10 countries
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Create a bar plot for the top 10 countries
sns.barplot(data=top_10_population_growth_2021, x='Population growth p
axes[0].set_xlabel('Population Growth Percent')
axes[0].set_ylabel('Country')
axes[0].set_title('Top 10 Countries with Highest Population Growth in

# Add data labels for the top 10 countries
for p in axes[0].patches:
    axes[0].annotate(f'{p.get_width():.2f}', (p.get_width(), p.get_y()

# Create a bar plot for the least 10 countries
sns.barplot(data=least_10_population_growth_2021, x='Population growth
axes[1].set_xlabel('Population Growth Percent')
axes[1].set_ylabel('Country')
axes[1].set_title('Least 10 Countries with Lowest Population Growth in

# Add data labels for the least 10 countries
for p in axes[1].patches:
    axes[1].annotate(f'{p.get_width():.2f}', (p.get_width(), p.get_y()

# Adjust layout and show the plots
plt.tight_layout()
plt.show()
```



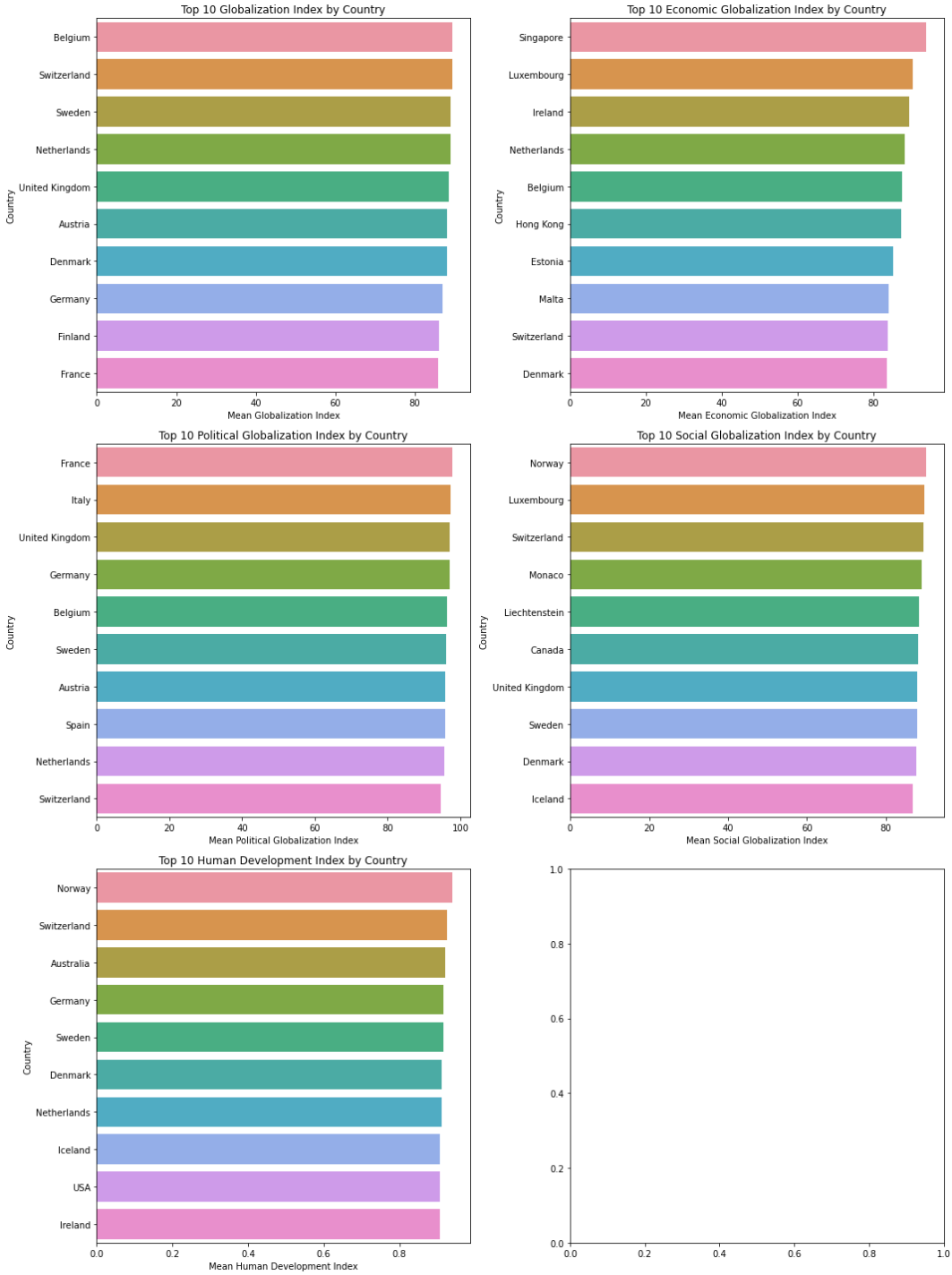
Observations

1. **High Population Growth**:
   - Countries like Niger, the Democratic Republic of the Congo, and Uganda experienced substantial population growth in 2021, with growth rates above 3%.
   - Factors contributing to this high growth include elevated birth rates, limited access to family planning, and potential healthcare and education challenges.
   - These countries are likely to face unique socio-economic and development challenges due to rapid population expansion.
2. **Negative Growth**:
   - Several nations, including Singapore, Croatia, and Qatar, witnessed negative population growth in 2021.
   - Reasons for this decline may include low birth rates, aging populations, and, in some cases, outmigration.
   - Negative population growth can pose challenges such as potential labor force shortages and impacts on social welfare and healthcare systems.
3. **Global Diversity**:
   - The varying population growth rates among countries in 2021 highlight the complex interplay of social, economic, and political factors.
   - Demographic dynamics differ widely from one region to another.
   - Policymakers and researchers need to recognize this diversity and develop tailored strategies to address the specific population challenges in different parts of the world.

Development Indices of - Top 10 Countries by their mean

In [35]:

```
# Create a function for the plot
def plot_globalization_indices(df, index_name, title, row, col):
    top_10 = df.groupby('Country')[index_name].mean().nlargest(10)

    # Create a subplot in a 3x2 grid
    sns.set_palette("Reds_r")
    sns.barplot(x=top_10.values, y=top_10.index, ax=axes[row, col])
    axes[row, col].set_xlabel(f'Mean {index_name}')
    axes[row, col].set_title(f'Top 10 {title} by Country')

# Create subplots in a 3x2 grid
fig, axes = plt.subplots(3, 2, figsize=(15, 20))

# Call the function for different indices
plot_globalization_indices(merged_df, 'Globalization Index', 'Globaliz
plot_globalization_indices(merged_df, 'Economic Globalization Index',
plot_globalization_indices(merged_df, 'Political Globalization Index',
plot_globalization_indices(merged_df, 'Social Globalization Index', 'S
plot_globalization_indices(merged_df, 'Human Development Index', 'Huma

# Adjust layout and show the plots
plt.tight_layout()
plt.show()
```



Observations

Globalization Index:

- Belgium, Switzerland, and Sweden have the highest mean Globalization Index, indicating strong global integration.
- These countries exhibit a high degree of economic, political, and social globalization.
- The Globalization Index offers a comprehensive view of a country's overall global integration.

Economic Globalization Index:

- Singapore tops the list in Economic Globalization, reflecting its robust economic connections.
- Luxembourg and Ireland follow closely, emphasizing their attractiveness for international businesses.
- The Economic Globalization Index assesses a country's openness to international trade and investment.

Political Globalization Index:

- France and Italy lead in Political Globalization, highlighting active involvement in international political affairs.
- The United Kingdom and Germany also rank high, indicating their influence in global politics.
- The Political Globalization Index signifies a country's political stability, international diplomacy and cooperation.

Social Globalization Index:

- Norway and Luxembourg are at the forefront of Social Globalization, emphasizing their openness to cross-cultural interactions.
- Canada and the United Kingdom also exhibit significant social globalization, indicating diverse and inclusive societies.
- The Social Globalization Index reflects a country's cultural openness and cross-cultural understanding.

Human Development Index:

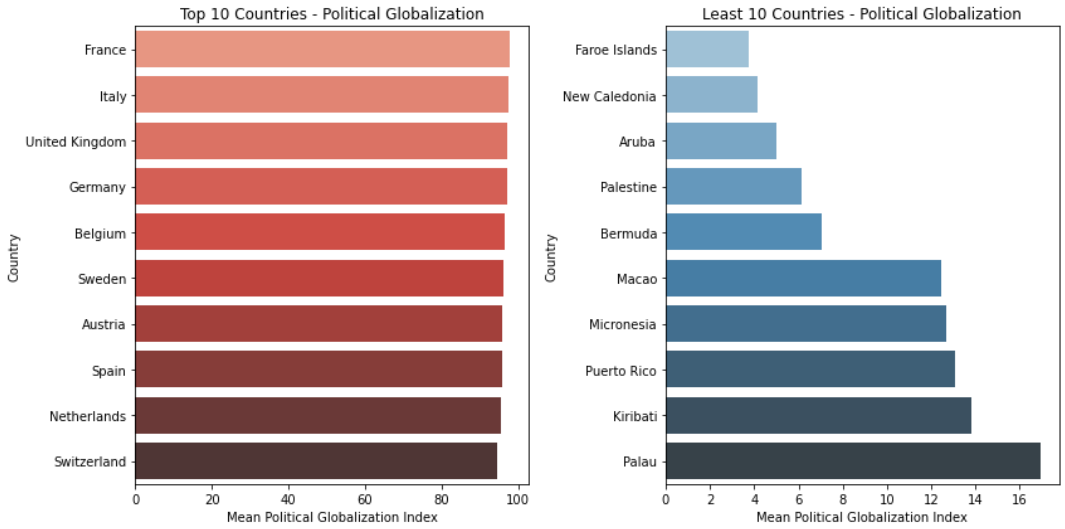
- Norway and Switzerland top the list for Human Development, emphasizing their commitment to quality of life, education, and well-being.
- These countries maintain high living standards, education, and healthcare, contributing to an excellent quality of life.
- The Human Development Index measures a nation's investment in well-being.

The countries identified as top performers in specific dimensions of globalization, such as economic, social, and human development indices, also demonstrate elevated levels of Political Globalization Index (PGI). This correlation suggests that their excellence in various globalized aspects is accompanied by a higher degree of political engagement on the global stage.

Political Globalization Index - Top 10 and Least 10 by their mean

In [36]:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate the mean Political Globalization Index by country
political_globalization_mean = merged_df.groupby('Country')['Political

# Get the top 10 countries with the highest mean Political Globalizati
top_10_political_globalization = political_globalization_mean.nlargest

# Get the least 10 countries with the lowest mean Political Globalizat
least_10_political_globalization = political_globalization_mean.nsmall

# Create a figure with two subplots
plt.figure(figsize=(12, 6))
plt.subplots_adjust(wspace=0.5)

# Plot the top 10 countries
plt.subplot(1, 2, 1)
sns.barplot(x=top_10_political_globalization.values, y=top_10_politica
plt.title("Top 10 Countries - Political Globalization")
plt.xlabel("Mean Political Globalization Index")
plt.ylabel("Country")

# Plot the least 10 countries
plt.subplot(1, 2, 2)
sns.barplot(x=least_10_political_globalization.values, y=least_10_poli
plt.title("Least 10 Countries - Political Globalization")
plt.xlabel("Mean Political Globalization Index")
plt.ylabel("Country")

# Display the graphs
plt.tight_layout()
plt.show()
```



Observations

Top 10 Countries by Mean Political Globalization Index:

1. **Active Global Political Players:** The top 10 countries with the highest mean Political Globalization Index, including France, Italy, and the United Kingdom, are active and influential participants in global politics. They engage in international diplomacy, negotiations, and collaborations, contributing to their significant political globalization scores.
2. **Internal Political Stability:** Internal political stability in these countries fosters an environment where policy makers can make informed and consistent decisions. It provides a foundation for them to engage in global diplomacy with a unified approach, as a politically stable nation is more likely to have a coherent foreign policy agenda.
3. **Contributors to Global Governance:** With high Political Globalization Index scores, these countries often play essential roles in international organizations and forums, addressing critical global issues such as climate change, human rights, and international security. Their active participation contributes to shaping the global political landscape.
4. **Global Diplomatic Reach:** These leading countries maintain diplomatic relations and partnerships with a wide range of nations. Their diplomatic networks span across various regions, allowing them to influence international political decisions and global governance structures effectively.

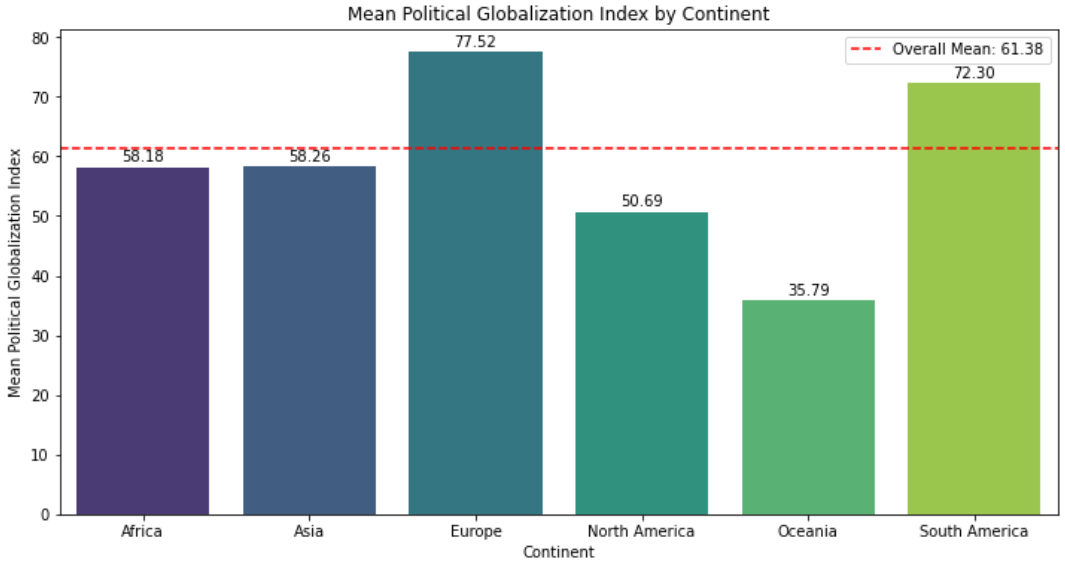
Least 10 Countries by Mean Political Globalization Index:

1. **Limited International Political Influence:** The least 10 countries with the lowest mean Political Globalization Index, such as Faroe Islands, New Caledonia, and Aruba, have limited international political influence. They are less engaged in global diplomatic activities and are not major players in international politics.
2. **Impact of Internal Political Stability:** Internal political stability, or the lack thereof, can significantly influence the foreign policy decisions of these countries. Political instability can lead to uncertainty and inconsistency in decision-making, affecting their ability to engage effectively in global politics.

3. **Potential for Increased Engagement:** While these countries have lower scores in political globalization, they have the opportunity to increase their international involvement and cooperation with other nations. Strengthening diplomatic relations and actively engaging in global discussions can help them enhance their political globalization status.
4. **Challenges in Diplomatic Participation:** These countries may face challenges in participating in international political affairs due to factors like their geographical

Mean Political Globalization Index - Continentwise

In [37]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Group the data by continent and calculate the mean Political Globali
mean_pgi_by_continent = merged_df.groupby('ContinentCode')['Political

# Calculate the overall mean Political Globalization Index
overall_mean_pgi = merged_df['Political Globalization Index'].mean()

# Create a bar chart for mean Political Globalization Index by contine
plt.figure(figsize=(12, 6))
bar_plot = sns.barplot(data=mean_pgi_by_continent, x='ContinentCode',

# Add labels with values
for index, value in enumerate(mean_pgi_by_continent['Political Globali
    plt.text(index, value + 0.5, f'{value:.2f}', ha='center', va='bott

# Add overall mean label
plt.axhline(y=overall_mean_pgi, color='red', linestyle='--', label=f'O
plt.title('Mean Political Globalization Index by Continent')
plt.xlabel('Continent')
plt.ylabel('Mean Political Globalization Index')
plt.legend()
plt.show()
```



- Average Political Globalization Index: 61.38

- These values provide a snapshot of the average political globalization scores for each continent, highlighting variations in political engagement and cooperation on a global scale. Europe followed by South America exhibits a higher level of political

Project Scope and Focus:

Project Focus:

- The primary focus of this project is an in-depth analysis of the "Political Globalization Index" within the dataset.

Specific Indicator:

- The central point of interest is the "Political Globalization Index." The analysis will uncover insights, trends, and correlations related to political globalization.

Exclusion of Other Indicators:

- To maintain focus on the political dimension, indicators such as "Economic Globalization Index," "Social Globalization Index," "Human Development Index," and additional demographic and healthcare indicators will be temporarily excluded from the analysis.

In [38]:
```
# List of columns to drop
columns_to_drop = ['Globalization Index', 'Economic Globalization Inde

# Drop the columns
merged_df = merged_df.drop(columns=columns_to_drop)
```

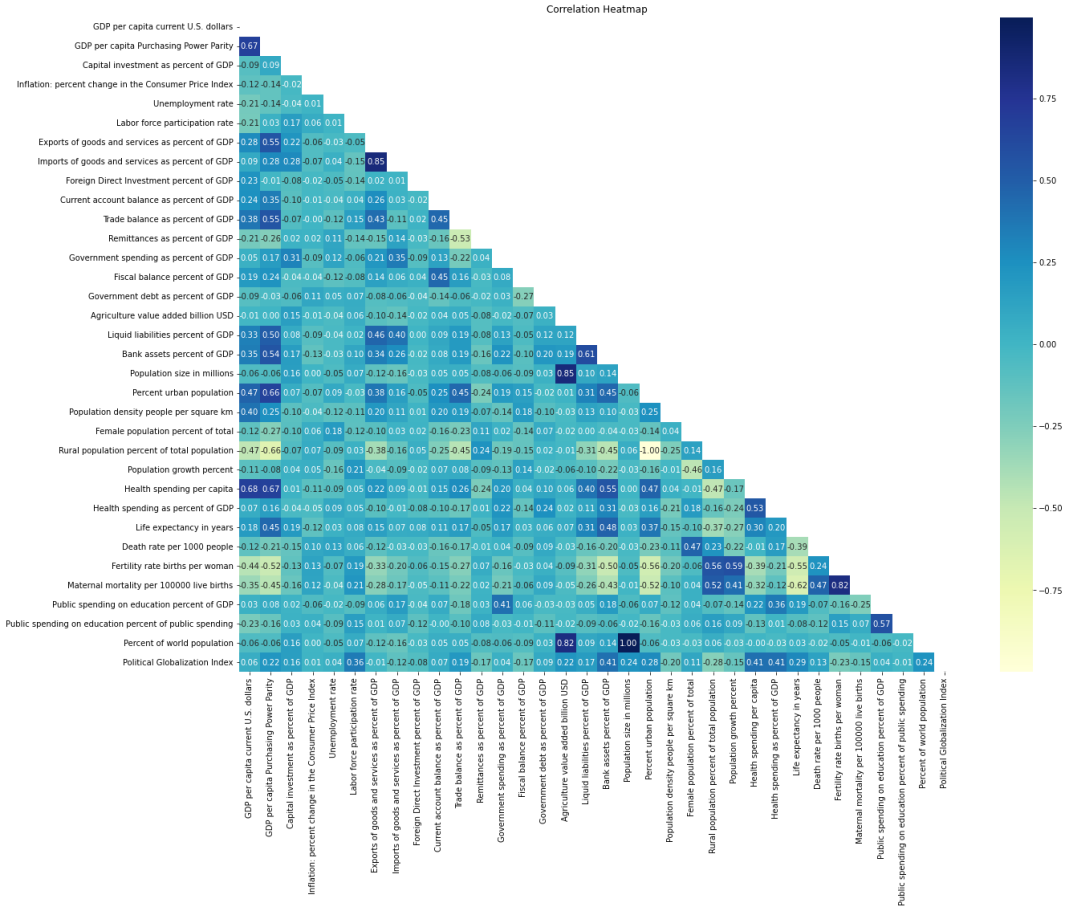
Correlation Matrix among Important features

In [39]:
```
merged_df.columns
```

Out[39]:
```
Index(['GDP per capita current U.S. dollars',
       'GDP per capita Purchasing Power Parity',
       'Capital investment as percent of GDP',
       'Inflation: percent change in the Consumer Price Index',
       'Unemployment rate', 'Labor force participation rate',
       'Exports of goods and services as percent of GDP',
       'Imports of goods and services as percent of GDP',
       'Foreign Direct Investment percent of GDP',
       'Current account balance as percent of GDP',
       'Trade balance as percent of GDP', 'Remittances as percent of
GDP',
       'Government spending as percent of GDP',
       'Fiscal balance percent of GDP', 'Government debt as percent o
f GDP',
       'Agriculture value added billion USD',
       'Liquid liabilities percent of GDP', 'Bank assets percent of G
DP',
       'Population size in millions', 'Percent urban population',
       'Population density people per square km',
       'Female population percent of total',
       'Rural population percent of total population',
       'Population growth percent', 'Health spending per capita',
       'Health spending as percent of GDP', 'Life expectancy in year
s',
       'Death rate per 1000 people', 'Fertility rate births per woma
n',
       'Maternal mortality per 100000 live births',
       'Public spending on education percent of GDP',
       'Public spending on education percent of public spending',
       'Percent of world population', 'Political Globalization Inde
x'],
      dtype='object')
```

In [40]:

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Assuming merged_df is a dataframe
correlation_matrix = merged_df.corr()

# Define a mask to hide the upper triangle for better visualization
mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))

# Create the heatmap
plt.figure(figsize=(20, 15))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="YlGnBu",
plt.title("Correlation Heatmap")
plt.show()
```



In [41]:
```
import pandas as pd
import hvplot.pandas

# Assuming merged_df is a dataframe
correlation_matrix = merged_df.corr()

# Select the correlation of 'Political Globalization Index' with other
correlation_with_political = correlation_matrix['Political Globalizati

# Visualize the correlations using hvplot
correlation_with_political.drop('Political Globalization Index').hvplo
    width=950, height=800,
    title="Correlation between Political Globalization Index and Numer
    ylabel='Correlation', xlabel='Numeric Features',
)
```

Out[41]:

Observations

Positive and Strong Correlation:

1. **Health Spending as Percent of GDP (0.411360):** There is a strong and positive correlation between the 'Political Globalization Index' and health spending as a percentage of GDP. This suggests that as a country's political globalization increases, it tends to allocate a higher portion of its GDP to healthcare. This could indicate that more politically globalized nations prioritize the well-being of their populations.

Positive and Moderate Correlation:

2. **Labor Force Participation Rate (0.355021):** A moderately positive correlation exists between the 'Political Globalization Index' and the labor force participation rate. This implies that more politically globalized countries tend to have a higher proportion of their population engaged in the workforce, reflecting active economic participation.
3. **Life Expectancy in Years (0.291281):** The positive correlation between political globalization and life expectancy is moderate. This suggests that politically globalized nations often experience longer life expectancies. This connection may be due to better healthcare infrastructure and living conditions in such countries.

Negative and Moderate Correlation:

4. **Rural Population Percent of Total Population (-0.278131):** A moderate negative correlation exists between the 'Political Globalization Index' and the rural population as a percentage of the total population. This implies that more politically globalized countries tend to have a lower proportion of their population residing in rural areas, reflecting urbanization trends.

5. **Fertility Rate (Births per Woman) (-0.229420):** The negative correlation between political globalization and fertility rate is moderate. This suggests that as a country becomes more politically globalized, its fertility rate tends to decrease. Factors like improved access to education and family planning services may contribute to this trend.
6. **Imports of Goods and Services as Percent of GDP (-0.121516):** There is a moderate negative correlation between political globalization and the share of imports in GDP. Politically globalized countries may place less emphasis on imports in their economies.

Negative and Weak Correlation:

7. **Imports of Goods and Services as Percent of GDP (-0.121516):** The weak negative correlation indicates that, while more politically globalized countries may have a slightly lower share of imports in GDP, this relationship is not very strong.
8. **Exports of Goods and Services as Percent of GDP (-0.008445):** The weak negative correlation with the share of exports in GDP suggests that there's only a subtle tendency for politically globalized countries to have a lower share of exports in their economies.

It's important to note that while these correlations provide insights, they do not imply causation. The strength of the correlation can vary, and multiple factors may influence

Applying Machine learning Models

In [42]:
```
from sklearn.model_selection import train_test_split


# Split the data into features (X) and target (y)
y = merged_df['Political Globalization Index']
X = merged_df.drop('Political Globalization Index', axis=1)

# Create a dictionary to store results
results = {}

# Split the data into training and testing sets (adjust the test_size
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
```

Linear Regression

Linear Regression is a foundational statistical and machine learning technique used to model relationships between a dependent variable (target) and independent variables (features). It employs a linear equation (Y = aX + b) to describe these relationships. The primary aim of Linear Regression is to determine the optimal linear equation (best-fitting line or hyperplane in multi-dimensional cases) that minimizes the squared differences between predicted and actual values, a method known as "least squares." This approach is valuable for both understanding relationships between variables and making predictions, and it serves as a fundamental tool in data analysis and predictive modeling.

In [43]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r


# Initialize the Linear Regression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Model evaluation
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Absolute Error: {mae:.4f}")
print(f"Mean Squared Error: {mse:.4f}")
print(f"R-squared (R2) Score: {r2:.4f}")

results['Linear Regression'] = [mae, mse, r2]
```

```
Mean Absolute Error: 12.8992
Mean Squared Error: 253.6423
R-squared (R2) Score: 0.5586
```

Decision Tree Regression

Decision Tree Regression is a machine learning technique that constructs a tree-like model to predict continuous target variables. It builds a tree structure by recursively splitting the data based on the most significant features, with each leaf node providing a prediction. Decision trees are highly interpretable, suitable for capturing non-linear relationships, robust against outliers, and widely applicable in various domains. They allow for feature importance assessment and are used as a foundation for powerful ensemble methods like Random Forests and Gradient Boosting, which address some of their limitations, including overfitting.

In [44]:
```
from sklearn.tree import DecisionTreeRegressor  # Import DecisionTreeR


# Initialize the Decision Tree Regressor model with limited depth
decision_tree_model = DecisionTreeRegressor(max_depth=50)  # Adjust th

# Fit the model to the training data
decision_tree_model.fit(X_train, y_train)

# Make predictions on the test data
y_pred_decision_tree = decision_tree_model.predict(X_test)

# Model evaluation
mae_decision_tree = mean_absolute_error(y_test, y_pred_decision_tree)
mse_decision_tree = mean_squared_error(y_test, y_pred_decision_tree)
r2_decision_tree = r2_score(y_test, y_pred_decision_tree)

# Print evaluation results
print("Decision Tree Regressor Evaluation:")
print(f"Mean Absolute Error: {mae_decision_tree:.2f}")
print(f"Mean Squared Error: {mse_decision_tree:.2f}")
print(f"R-squared (R2) Score: {r2_decision_tree:.2f}")

results['Decision Tree Regression'] = [mae_decision_tree, mse_decision
```

```
Decision Tree Regressor Evaluation:
Mean Absolute Error: 2.74
Mean Squared Error: 33.52
R-squared (R2) Score: 0.94
```

Random Forest Regressor

In [45]:
```
from sklearn.ensemble import RandomForestRegressor

# Initialize the Random Forest Regressor model
random_forest_model = RandomForestRegressor(n_estimators=100, random_s

# Fit the model to the training data
random_forest_model.fit(X_train, y_train)

# Make predictions on the test data
y_pred_random_forest = random_forest_model.predict(X_test)

# Model evaluation
mae_random_forest = mean_absolute_error(y_test, y_pred_random_forest)
mse_random_forest = mean_squared_error(y_test, y_pred_random_forest)
r2_random_forest = r2_score(y_test, y_pred_random_forest)

# Print evaluation results
print("Random Forest Regressor Evaluation:")
print(f"Mean Absolute Error: {mae_random_forest:.2f}")
print(f"Mean Squared Error: {mse_random_forest:.2f}")
print(f"R-squared (R2) Score: {r2_random_forest:.2f}")

results['Decision Tree Regression'] = [mae_random_forest, mse_random_f
```

```
Random Forest Regressor Evaluation:
Mean Absolute Error: 2.20
Mean Squared Error: 13.24
R-squared (R2) Score: 0.98
```

XGBoost Regressor

In [46]:
```
#pip install xgboost
```

In [47]:

```
import xgboost as xgb

# Initialize the XGBoost Regressor model
xgb_model = xgb.XGBRegressor(objective="reg:squarederror", n_estimator

# Fit the model to the training data
xgb_model.fit(X_train, y_train)

# Make predictions on the test data
y_pred_xgb = xgb_model.predict(X_test)

# Model evaluation
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)

# Print evaluation results
print("XGBoost Regressor Evaluation:")
print(f"Mean Absolute Error: {mae_xgb:.2f}")
print(f"Mean Squared Error: {mse_xgb:.2f}")
print(f"R-squared (R2) Score: {r2_xgb:.2f}")

results['XGBoost Regressor'] = [mae_xgb, mse_xgb, r2_xgb]
```

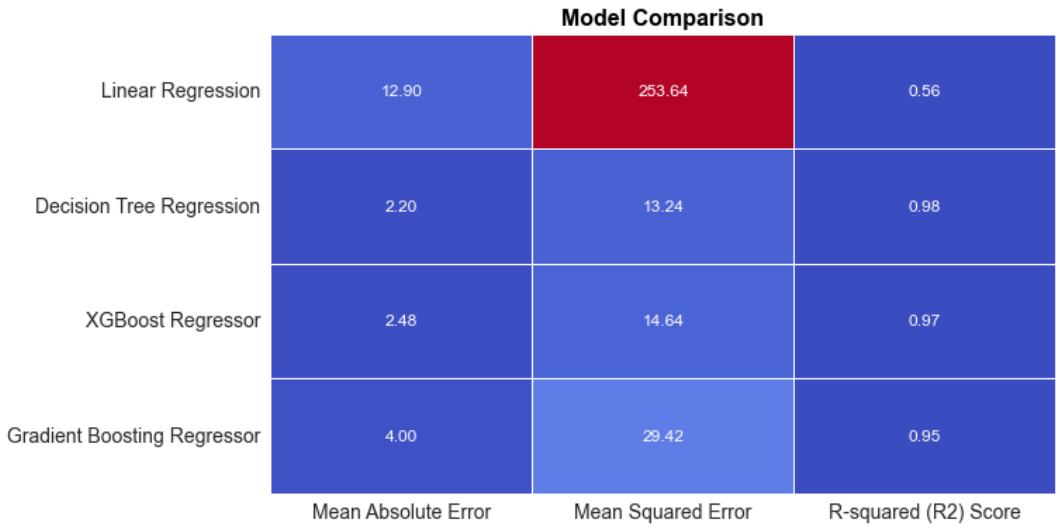
```
XGBoost Regressor Evaluation:
Mean Absolute Error: 2.48
Mean Squared Error: 14.64
R-squared (R2) Score: 0.97
```

Gradient Boosting Regressor

In [48]:
```
from sklearn.ensemble import GradientBoostingRegressor

# Initialize the Gradient Boosting Regressor model
gradient_boosting_model = GradientBoostingRegressor(n_estimators=100,

# Fit the model to the training data
gradient_boosting_model.fit(X_train, y_train)

# Make predictions on the test data
y_pred_gradient_boosting = gradient_boosting_model.predict(X_test)

# Model evaluation
mae_gradient_boosting = mean_absolute_error(y_test, y_pred_gradient_bo
mse_gradient_boosting = mean_squared_error(y_test, y_pred_gradient_boo
r2_gradient_boosting = r2_score(y_test, y_pred_gradient_boosting)

# Print evaluation results
print("Gradient Boosting Regressor Evaluation:")
print(f"Mean Absolute Error: {mae_gradient_boosting:.2f}")
print(f"Mean Squared Error: {mse_gradient_boosting:.2f}")
print(f"R-squared (R2) Score: {r2_gradient_boosting:.2f}")

results['Gradient Boosting Regressor'] = [mae_gradient_boosting, mse_g
```

```
Gradient Boosting Regressor Evaluation:
Mean Absolute Error: 4.00
Mean Squared Error: 29.42
R-squared (R2) Score: 0.95
```

In [49]:
```
# Create a pandas DataFrame from the results dictionary
results_df = pd.DataFrame.from_dict(results, orient='index', columns=[

# Set a different style for the plot
sns.set_style("darkgrid")

# Increase font size for labels and annotations
label_fontsize = 14  # Adjust the label font size as needed
annot_fontsize = 12  # Adjust the annotation font size as needed

# Plot the table using seaborn
plt.figure(figsize=(10, 6))
ax = sns.heatmap(results_df, annot=True, cmap="coolwarm", fmt=".2f", l
ax.set_xticklabels(ax.get_xticklabels(), fontsize=label_fontsize)  # I
ax.set_yticklabels(ax.get_yticklabels(), fontsize=label_fontsize)  # I

# Increase title font size, make it bold, and set the color to black
plt.title("Model Comparison", fontsize=16, fontweight='bold', color='b

plt.show()
```



Based on the mertics, we can conclude that Random Forest Regressor has the best performance with 0.98 R2 value, with 13.24 of MSE value and with 2.20 of MAE

Applying GridSearchCV for Hyperparameter tuning for Regression Models

GridSearchCV (Grid Search Cross-Validation) and hyperparameter tuning are essential techniques in machine learning for improving the performance of models. They play a critical role in finding the optimal set of hyperparameters for a model to achieve the best

results. Let's explore the importance of GridSearchCV and the variables used in `param_grid` during hyperparameter tuning:

Importance of GridSearchCV and Hyperparameter Tuning:

1. **Optimizing Model Performance:** The primary objective of hyperparameter tuning is to find the hyperparameters that yield the best model performance on your dataset. It helps improve accuracy, reduce overfitting, and enhance generalization.
2. **Avoiding Overfitting:** Hyperparameters control the complexity of a model. By fine-tuning them, you can prevent overfitting (where a model learns the training data too well but struggles to generalize to new data) and achieve a better trade-off between bias and variance.
3. **Generalization:** A model with well-tuned hyperparameters is more likely to generalize well to unseen data. It's essential for ensuring that your machine learning model performs well in real-world scenarios.
4. **Reducing Training Time:** Hyperparameter tuning helps identify the most efficient set of hyperparameters, which can reduce the time it takes to train a model. This is particularly important when working with large datasets and complex models.
5. **Model Interpretability:** Certain hyperparameters can impact the interpretability of a model. For example, the depth of a decision tree in a random forest can affect the complexity and transparency of the model.

Now, let's discuss the variables typically used in the `param_grid` dictionary during hyperparameter tuning:

- **n_estimators:** The number of base estimators (trees in the case of ensemble methods like Random Forest or Gradient Boosting).
- **learning_rate:** The step size at each iteration while moving toward a minimum of a loss function in gradient boosting models.
- **max_depth:** The maximum depth of decision trees in decision tree-based algorithms.
- **min_samples_split:** The minimum number of samples required to split an internal node in decision tree-based algorithms.
- **min_samples_leaf:** The minimum number of samples required to be at a leaf node in decision tree-based algorithms.
- **subsample:** The fraction of samples used for fitting the trees in ensemble methods like Gradient Boosting.
- **C:** The regularization parameter in linear models like Support Vector Machines (SVM), which controls the trade-off between fitting the training data and preventing overfitting.
- **epsilon:** A parameter in Support Vector Regression (SVR) that specifies the margin of tolerance where no penalty is given to errors.
- **gamma:** A parameter in SVM and some other models, determining the shape of the decision boundary.

These variables impact the model's behavior and complexity. For example, increasing `max_depth` in a decision tree can lead to a more complex model, while a higher `C` value in SVM results in a less regularized model. The choice of values for these variables depends on the dataset and the specific problem, and GridSearchCV helps you systematically explore a range of values to find the best combination.

Hyperparameter tuning, facilitated by GridSearchCV, is crucial for building robust and high-performing machine learning models that are well-suited to the specific problem at hand.

Support Vector Regressor - Hyper Parameter Tuning using GridSearchCV

In [50]:
```
# Create a dictionary to store results
results_GridSearchCV = {}
```

In [64]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVR

param_grid = {
    'kernel': ['rbf', 'poly'],
    'C': [0.1, 1, 10],
}


grid_search = GridSearchCV(SVR(), param_grid, cv=5, n_jobs=-1)
grid_search.fit(X_train, y_train)

best_svr_model = grid_search.best_estimator_

y_pred_best_svr = best_svr_model.predict(X_test)

# Model evaluation
mae_svr = mean_absolute_error(y_test, y_pred_best_svr)
mse_svr = mean_squared_error(y_test, y_pred_best_svr)
r2_svr = r2_score(y_test, y_pred_best_svr)

# Print evaluation results
print("Support Vector Regressor (SVR) Evaluation:")
print(f"Mean Absolute Error: {mae_svr:.2f}")
print(f"Mean Squared Error: {mse_svr:.2f}")
print(f"R-squared (R2) Score: {r2_svr:.2f}")

results['Support Vector Regressor'] = [mae_svr, mse_svr, r2_svr]
```

```
Support Vector Regressor (SVR) Evaluation:
Mean Absolute Error: 15.35
Mean Squared Error: 407.77
R-squared (R2) Score: 0.28
```

Random Forest Regressor - Hyper Parameter Tuning using GridSearchCV

In [52]:
```
from sklearn.model_selection import GridSearchCV

# Initialize the Random Forest Regressor model
random_forest_model = RandomForestRegressor(random_state=42)

# Define a reduced set of hyperparameters and their possible values to
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# Create a GridSearchCV object with a smaller number of cross-validati
grid_search = GridSearchCV(random_forest_model, param_grid, cv=3, scor

# Fit the GridSearchCV object to the training data
grid_search.fit(X_train, y_train)

# Get the best parameters and the best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

# Make predictions on the test data using the best model
y_pred_random_forest_best = best_model.predict(X_test)

# Model evaluation with the best model
mae_random_forest_best = mean_absolute_error(y_test, y_pred_random_for
mse_random_forest_best = mean_squared_error(y_test, y_pred_random_fore
r2_random_forest_best = r2_score(y_test, y_pred_random_forest_best)

# Print best parameters and evaluation results
print("Best Hyperparameters:")
print(best_params)
print("\nRandom Forest Regressor Evaluation with Best Model:")
print(f"Mean Absolute Error: {mae_random_forest_best:.2f}")
print(f"Mean Squared Error: {mse_random_forest_best:.2f}")
print(f"R-squared (R2) Score: {r2_random_forest_best:.2f}")

results_GridSearchCV['Random Forest Regressor'] = [mae_random_forest_b
```

```
Best Hyperparameters:
{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n
_estimators': 100}

Random Forest Regressor Evaluation with Best Model:
Mean Absolute Error: 2.20
Mean Squared Error: 13.24
R-squared (R2) Score: 0.98
```

Decision Tree Regressor - Hyper Parameter Tuning using GridSearchCV

In [66]:

```
# Initialize the Decision Tree Regressor model
decision_tree_model = DecisionTreeRegressor()

# Define a smaller set of hyperparameters and their possible values
param_grid = {
    'max_depth': [10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# Create a GridSearchCV object with mean squared error as the scoring
grid_search = GridSearchCV(
    decision_tree_model, param_grid, cv=5, scoring='neg_mean_squared_e
)

# Fit the GridSearchCV object to the training data
grid_search.fit(X_train, y_train)

# Get the best hyperparameters and the best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

# Make predictions on the test data using the best model
y_pred_decision_tree_best = best_model.predict(X_test)

# Model evaluation with the best model
mae_decision_tree_best = mean_absolute_error(y_test, y_pred_decision_t
mse_decision_tree_best = mean_squared_error(y_test, y_pred_decision_tr
r2_decision_tree_best = best_model.score(X_test, y_test)  # R2 score

# Print best hyperparameters and evaluation results
print("Best Hyperparameters:")
print(best_params)
print("\nDecision Tree Regressor Evaluation with Best Model:")
print(f"Mean Absolute Error: {mae_decision_tree_best:.2f}")
print(f"Mean Squared Error: {mse_decision_tree_best:.2f}")
print(f"R-squared (R2) Score: {r2_decision_tree_best:.2f}")

results_GridSearchCV['Decision Tree Regressor'] = [mae_decision_tree_b
```

```
Best Hyperparameters:
{'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 5}

Decision Tree Regressor Evaluation with Best Model:
Mean Absolute Error: 2.68
Mean Squared Error: 28.25
R-squared (R2) Score: 0.95
```

XGBoost Regressor - Hyper Parameter Tuning using GridSearchCV

In [67]:

```
import xgboost as xgb


# Initialize the XGBoost Regressor model
xgb_model = xgb.XGBRegressor(objective="reg:squarederror", n_estimator

# Define a reduced set of hyperparameters and their possible values to
param_grid = {
    'n_estimators': [100, 200],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 4],
    'min_child_weight': [1, 2],
    'gamma': [0, 0.1],
    'subsample': [0.8, 0.9],
    'colsample_bytree': [0.8, 0.9]
}

# Create a GridSearchCV object with a smaller number of cross-validati
grid_search = GridSearchCV(xgb_model, param_grid, cv=3, scoring='neg_m

# Fit the GridSearchCV object to the training data
grid_search.fit(X_train, y_train)

# Get the best parameters and the best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

# Make predictions on the test data using the best model
y_pred_xgb_best = best_model.predict(X_test)

# Model evaluation with the best model
mae_xgb_best = mean_absolute_error(y_test, y_pred_xgb_best)
mse_xgb_best = mean_squared_error(y_test, y_pred_xgb_best)
r2_xgb_best = r2_score(y_test, y_pred_xgb_best)

# Print best parameters and evaluation results
print("Best Hyperparameters:")
print(best_params)
print("\nXGBoost Regressor Evaluation with Best Model:")
print(f"Mean Absolute Error: {mae_xgb_best:.2f}")
print(f"Mean Squared Error: {mse_xgb_best:.2f}")
print(f"R-squared (R2) Score: {r2_xgb_best:.2f}")

results_GridSearchCV['XGBoost Regressor'] = [mae_xgb_best, mse_xgb_bes
```

```
Best Hyperparameters:
{'colsample_bytree': 0.8, 'gamma': 0.1, 'learning_rate': 0.1, 'max_de
pth': 4, 'min_child_weight': 2, 'n_estimators': 200, 'subsample': 0.
8}

XGBoost Regressor Evaluation with Best Model:
Mean Absolute Error: 2.68
Mean Squared Error: 13.94
R-squared (R2) Score: 0.98
```

Gradient Regressor - Hyper Parameter Tuning using GridSearchCV

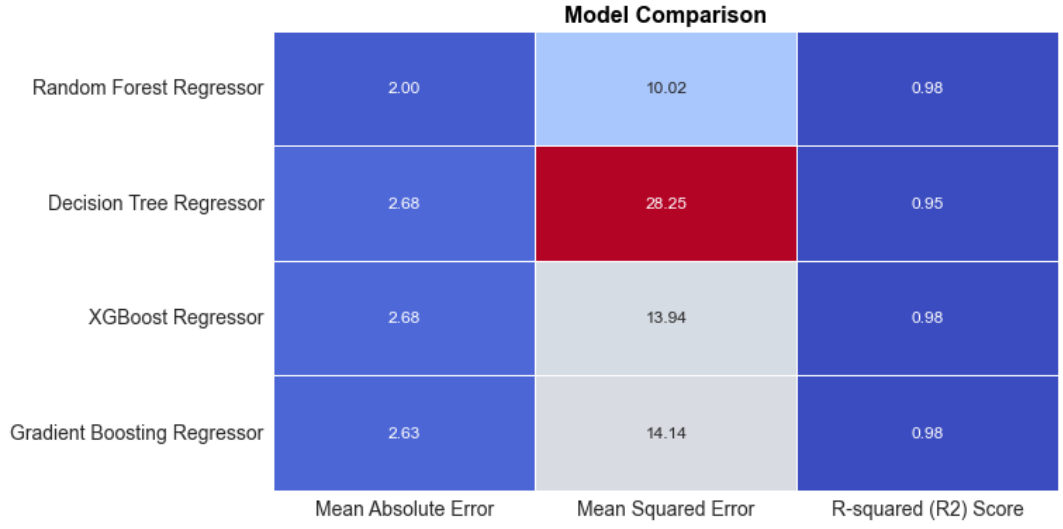
In [68]:

```
from sklearn.ensemble import GradientBoostingRegressor


# Initialize the Gradient Boosting Regressor model
gradient_boosting_model = GradientBoostingRegressor(n_estimators=100,

# Define a reduced set of hyperparameters and their possible values to
param_grid = {
    'n_estimators': [100, 200],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 4],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'subsample': [0.8, 0.9]
}

# Create a GridSearchCV object with a smaller number of cross-validati
grid_search = GridSearchCV(gradient_boosting_model, param_grid, cv=3,

# Fit the GridSearchCV object to the training data
grid_search.fit(X_train, y_train)

# Get the best parameters and the best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

# Make predictions on the test data using the best model
y_pred_gradient_boosting_best = best_model.predict(X_test)

# Model evaluation with the best model
mae_gradient_boosting_best = mean_absolute_error(y_test, y_pred_gradie
mse_gradient_boosting_best = mean_squared_error(y_test, y_pred_gradien
r2_gradient_boosting_best = r2_score(y_test, y_pred_gradient_boosting_

# Print best parameters and evaluation results
print("Best Hyperparameters:")
print(best_params)
print("\nGradient Boosting Regressor Evaluation with Best Model:")
print(f"Mean Absolute Error: {mae_gradient_boosting_best:.2f}")
print(f"Mean Squared Error: {mse_gradient_boosting_best:.2f}")
print(f"R-squared (R2) Score: {r2_gradient_boosting_best:.2f}")

results_GridSearchCV['Gradient Boosting Regressor'] = [mae_gradient_bo
```

```
Best Hyperparameters:
{'learning_rate': 0.1, 'max_depth': 4, 'min_samples_leaf': 2, 'min_sa
mples_split': 2, 'n_estimators': 200, 'subsample': 0.8}

Gradient Boosting Regressor Evaluation with Best Model:
Mean Absolute Error: 2.63
Mean Squared Error: 14.14
R-squared (R2) Score: 0.98
```

In [69]:

```
# Create a pandas DataFrame from the results dictionary
results_df = pd.DataFrame.from_dict(results_GridSearchCV, orient='inde

# Set a different style for the plot
sns.set_style("darkgrid")

# Increase font size for labels and annotations
label_fontsize = 14  # Adjust the label font size as needed
annot_fontsize = 12  # Adjust the annotation font size as needed

# Plot the table using seaborn
plt.figure(figsize=(10, 6))
ax = sns.heatmap(results_df, annot=True, cmap="coolwarm", fmt=".2f", l
ax.set_xticklabels(ax.get_xticklabels(), fontsize=label_fontsize)  # I
ax.set_yticklabels(ax.get_yticklabels(), fontsize=label_fontsize)  # I

# Increase title font size, make it bold, and set the color to black
plt.title("Model Comparison", fontsize=16, fontweight='bold', color='b

plt.show()
```



In [ ]:

```
```

Proto Type Steps

Choosen the Best Model based on the Results (metrics - 'Mean Absolute Error', 'Mean Squared Error', 'R-squared (R2) Score') mode

The best Model is RandomForestRegressor with Hyper Parameter Tuning

Best Hyperparameters:

{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}

In [53]:

```
# Set the best hyperparameters
best_hyperparameters = {
    'max_depth': None,
    'min_samples_leaf': 1,
    'min_samples_split': 2,
    'n_estimators': 100
}

# Create the RandomForestRegressor with the best hyperparameters
best_RandomForest_model = RandomForestRegressor(**best_hyperparameters

# Fit the model to the training data
best_RandomForest_model.fit(X_train, y_train)

# Make predictions on the test data
predictions = best_RandomForest_model.predict(X_test)

# Evaluate the model using mean squared error
mse = mean_squared_error(y_test, predictions)
print(f'Mean Squared Error on Test Data: {mse}')

# Calculate and print R-squared
r2 = r2_score(y_test, predictions)
print(f'R-squared on Test Data: {r2}')
```

```
Mean Squared Error on Test Data: 13.033411190518796
R-squared on Test Data: 0.9773186176360108
```

The above Model is saved to model.pkl

In [55]:
```
import joblib

model_filename = "C://Users//Rajashekhar//Desktop//test//model.pkl"
joblib.dump(best_RandomForest_model, model_filename)
```

Out[55]:
```
['C://SPU//Sandhya//Capstone//model.pkl']
```

In [ ]:
```

```

ProtoType Code is written in app.py

In [1]:

```
# app.py

import joblib
import streamlit as st
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Load the trained model
loaded_model = joblib.load('model.pkl')

# Streamlit App
st.title("PGI Prediction")

# Sidebar with user input
st.sidebar.header("User Input Features")

# Get user input
GDP_USdollars = st.sidebar.number_input("GDP per capita current U.S. d
GDP_PPP = st.sidebar.number_input("GDP per capita Purchasing Power Par
Capital_investment = st.sidebar.number_input("Capital investment as pe
Inflation = st.sidebar.number_input("Inflation: percent change in the
Unemployment_rate = st.sidebar.number_input("Unemployment rate")
Labor_force_participation_rate = st.sidebar.number_input("Labor force
Exports_of_goods_and_services_as_percent_of_GDP = st.sidebar.number_in
Imports_of_goods_and_services_as_percent_of_GDP = st.sidebar.number_in
Foreign_Direct_Investment_percent_of_GDP = st.sidebar.number_input("Fo
Current_account_balance_as_percent_of_GDP = st.sidebar.number_input("C
Trade_balance_as_percent_of_GDP = st.sidebar.number_input("Trade balan
Remittances_as_percent_of_GDP = st.sidebar.number_input("Remittances a
Government_spending_as_percent_of_GDP = st.sidebar.number_input("Gover
Fiscal_balance_percent_of_GDP = st.sidebar.number_input("Fiscal balanc
Government_debt_as_percent_of_GDP = st.sidebar.number_input("Governmen
Agriculture_value_added_billion_USD = st.sidebar.number_input("Agricul
Liquid_liabilities_percent_of_GDP = st.sidebar.number_input("Liquid li
Bank_assets_percent_of_GDP = st.sidebar.number_input("Bank assets perc
Population_size_in_millions = st.sidebar.number_input("Population size
Percent_urban_population = st.sidebar.number_input("Percent urban popu
Population_density_people_per_square_km = st.sidebar.number_input("Pop
Female_population_percent_of_total = st.sidebar.number_input("Female p
Rural_population_percent_of_total_population = st.sidebar.number_input
Population_growth_percent = st.sidebar.number_input("Population growth
Health_spending_per_capita = st.sidebar.number_input("Health spending
Health_spending_as_percent_of_GDP = st.sidebar.number_input("Health sp
Life_expectancy_in_years = st.sidebar.number_input("Life expectancy in
Death_rate_per_1000_people = st.sidebar.number_input("Death rate per 1
Fertility_rate_births_per_woman = st.sidebar.number_input("Fertility r
Maternal_mortality_per_100000_live_births = st.sidebar.number_input("M
Public_spending_on_education_percent_of_GDP = st.sidebar.number_input(
Public_spending_on_education_percent_of_public_spending = st.sidebar.n
Percent_of_world_population = st.sidebar.number_input("Percent of worl


# Create a DataFrame with the user input
user_input = pd.DataFrame({
    'GDP_USdollars': [GDP_USdollars],
    'GDP_PPP': [GDP_PPP],
    'Capital_investment': [Capital_investment],
```

```
    'Inflation': [Inflation],
    'Unemployment_rate': [Unemployment_rate],
    'Labor_force_participation_rate': [Labor_force_participation_rate]
    'Exports_of_goods_and_services_as_percent_of_GDP': [Exports_of_goo
    'Imports_of_goods_and_services_as_percent_of_GDP': [Imports_of_goo
    'Foreign_Direct_Investment_percent_of_GDP': [Foreign_Direct_Invest
    'Current_account_balance_as_percent_of_GDP': [Current_account_bala
    'Trade_balance_as_percent_of_GDP': [Trade_balance_as_percent_of_GD
    'Remittances_as_percent_of_GDP': [Remittances_as_percent_of_GDP],
    'Government_spending_as_percent_of_GDP': [Government_spending_as_p
    'Fiscal_balance_percent_of_GDP': [Fiscal_balance_percent_of_GDP],
    'Government_debt_as_percent_of_GDP': [Government_debt_as_percent_o
    'Agriculture_value_added_billion_USD': [Agriculture_value_added_bi
    'Liquid_liabilities_percent_of_GDP': [Liquid_liabilities_percent_o
    'Bank_assets_percent_of_GDP': [Bank_assets_percent_of_GDP],
    'Population_size_in_millions': [Population_size_in_millions],
    'Percent_urban_population': [Percent_urban_population],
    'Population_density_people_per_square_km': [Population_density_peo
    'Female_population_percent_of_total': [Female_population_percent_o
    'Rural_population_percent_of_total_population': [Rural_population_
    'Population_growth_percent': [Population_growth_percent],
    'Health_spending_per_capita': [Health_spending_per_capita],
    'Health_spending_as_percent_of_GDP': [Health_spending_as_percent_o
    'Life_expectancy_in_years': [Life_expectancy_in_years],
    'Death_rate_per_1000_people': [Death_rate_per_1000_people],
    'Fertility_rate_births_per_woman': [Fertility_rate_births_per_woma
    'Maternal_mortality_per_100000_live_births': [Maternal_mortality_p
    'Public_spending_on_education_percent_of_GDP': [Public_spending_on
    'Public_spending_on_education_percent_of_public_spending': [Public
    'Percent_of_world_population': [Percent_of_world_population]
})

# Ensure that the user input has the same data types as the training d
user_input = user_input.astype(float)

# Make predictions
prediction = loaded_model.predict(user_input)

# Display the prediction
st.subheader("Predicted PGI:")
st.write(prediction)
```

In [ ]:

```
streamlit run app.py
```